



Python for Introductory Statistics Lab Workbook

Mathematics Department, Fall 2021, Google Colab Edition

Student's Lab Workbook: A companion to [Python for Introductory Statistics](#)

Questions, comments or errors can be reported by sending an email to saman2@ccc.edu. A link to an interactive Colab version of this document can be found in this [interactive Colab workbook](#).

Keyboard Shortcut for HTML (Weblink) and PDF Users: When you click an external link and want to return to where you were before the click, use `Alt + Left Arrow` on Windows and `Command + Left Arrow` on Mac.

© 2021 Simon Aman, City Colleges of Chicago. Special thanks to reviewers, Professor Phil Yates of DePaul University and Professor Aman Ghebremicael of Arizona Western College.

This work is licensed under a [Creative Commons Attribution 4.0 International License \(CC BY 4.0\)](#).

Table of Contents

1. Introduction to Python

- 1.1 Getting Started
- 1.2 Python Arithmetic Operators
- 1.3 Python Variables
- 1.4 Python Comments
- 1.5 The Numpy Library
- 1.6 `input` Data From Keyboard

2. Python Lists

- 2.1 Creating and Accessing Lists
- 2.2 List Operations

3. Python Syntax

- 3.1 Python `if` statements
- 3.2 Python functions
- 3.3 Syntax Errors

4. Python Loops

- 4.1 The `for` Loop
- 4.2 The `while` Loop

5. Frequency Distribution and Graphs

- 5.1 Frequency Distributions
- 5.3 Bar Graphs and Pie Charts
- 5.4 Histogram, Polygon and Ogive
- 5.5 Dot Plot
- 5.5 Stem and Leaf Plot

6. Measures of Center and Variation

- 6.1 Mean and Median
- 6.2 Variance and Standard Deviation
- 6.4 Grouped Mean and Standard Deviation
- 7. Measures of Position
 - 7.1 Percentiles
 - 7.2 SciPy and Percentile Rank
 - 7.3 Box Plot
- 8. Discrete Probability Distributions
 - 8.1 Expected Values and Standard Deviations
 - 8.2 Binomial Probability Distribution
- 9. Continuous Probability Distribution
 - 9.1 Uniform Probability Distribution
 - 9.2 Normal Probability Distribution
 - 9.3 Central Limit Theorem: Mean
- 10. Confidence Intervals
 - 10.1 Confidence Interval for Mean (z distribution)
 - 10.2 Confidence Interval for Mean (t distribution)
 - 10.3 Confidence Interval for Proportions
- 11. Hypothesis Testing
 - 11.1 Hypothesis Testing: Mean z test
 - 11.2 Hypothesis Testing: Mean t test
 - 11.3 Hypothesis Testing: Proportion z test
- 12. Linear Regression and Correlation
 - 12.1 Scatter Plot
 - 12.2 Correlation
 - 12.3 Linear Regression
- 13. Analysis of Categorical Data
 - 13.1 Goodness of Fit
 - 13.2 Test of Independence
- 14. Advanced Topics
 - 14.1 Descriptive Statistics with Pandas
 - 14.2 Uploading Excel Data
 - 14.3 Visualization
 - 14.4 Regression

- [14.5 ANOVA](#)
 - [Lab Answers](#)
 - [Appendix 1](#)
 - [Appendix 2](#)
 - [Appendix 3](#)
 - [References](#)
 - [Resources Link](#)
-

1. Introduction to Python

Python is a general-purpose simple-syntax programming language created by Guido van Rossum in 1989. It is used in scientific computing, data science, web development, software development, artificial intelligence, machine learning etc.

1.1. Getting Started

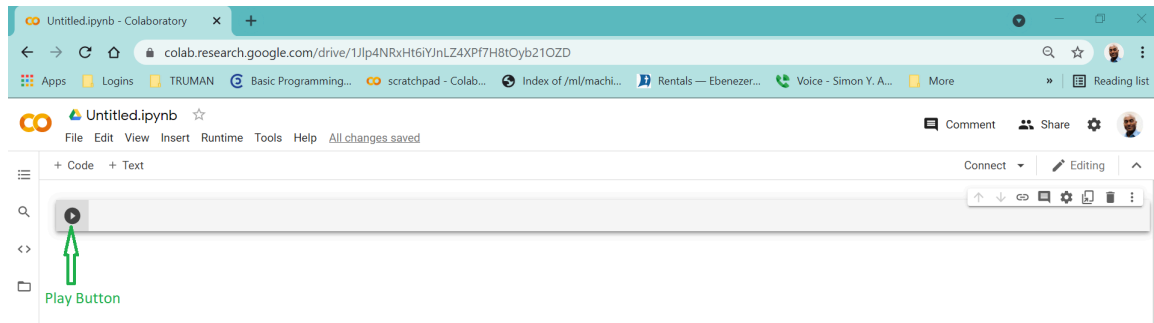
Google Colaboratory Notebook (**Colab** for short) is a free web-based Google product, similar to Google Docs, that requires no software installation for Python programming. A Colab Notebook is a list of **cells** created by adding a **code** or a **text** cell. When you create your own Colab notebooks, they are stored in your Google Drive account.

To Open a New Colab Notebook

[Watch demo video](#)

1. Click [this link](#).
2. Sign in with your Google account. If you are already signed in, continue to the next step. You can create a free [Google account](#).
3. Click **New Notebook** from the bottom-right of the pop-up window. Note: It may open a new notebook automatically sometimes.

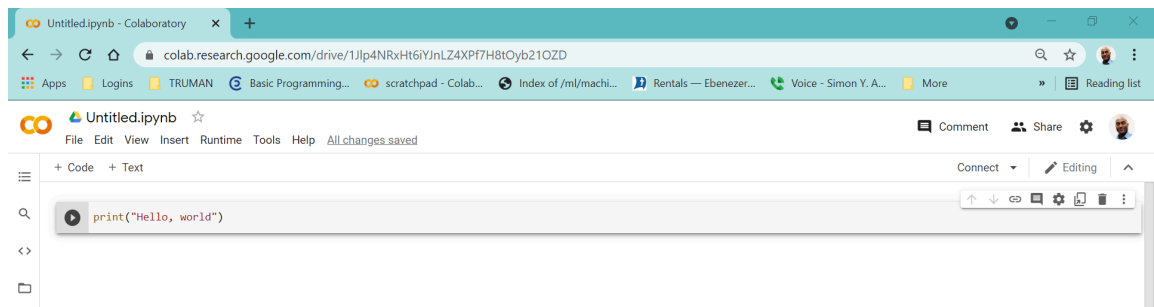
Note: Google Colab Notebook (Picture Below)



To Write Your First Python Code

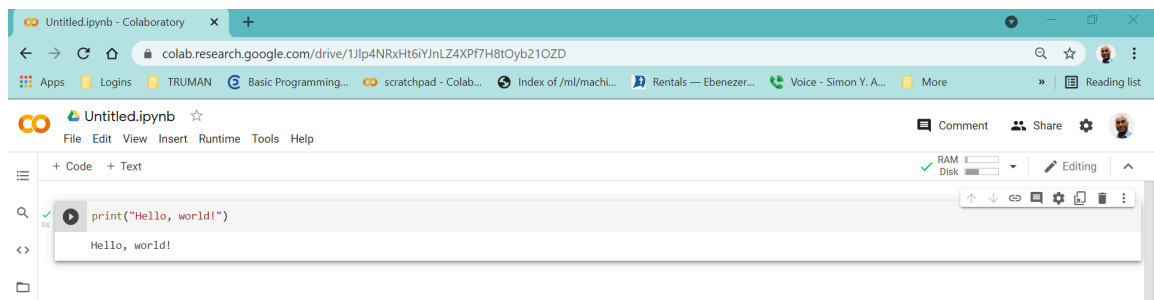
1. Click the code-cell next to the `play` button
2. Type `print("Hello, World!")`

See picture below



To Execute(run) Your First Python Code

1. Click the `play` button located before `print("Hello, World!")`
2. Or click on your first Python code and press Ctrl and Enter on your keyboard. Command and Enter for Mac(Apple) computers.
3. `Hello, world!` is displayed on a new line



To Edit the Code

1. Click the code-cell with `print("Hello, World!")`
2. Start editing

To Write Your Second Python Code

1. Click **+ Code** in your notebook
2. Type **10+10** in the new code cell
3. Click the **play** button
4. The output **20** is displayed on a new line



Note: If you prefer **not** to use Google Colab Notebook, you can install free Python in your computer. See [appendix 1](#) for details.

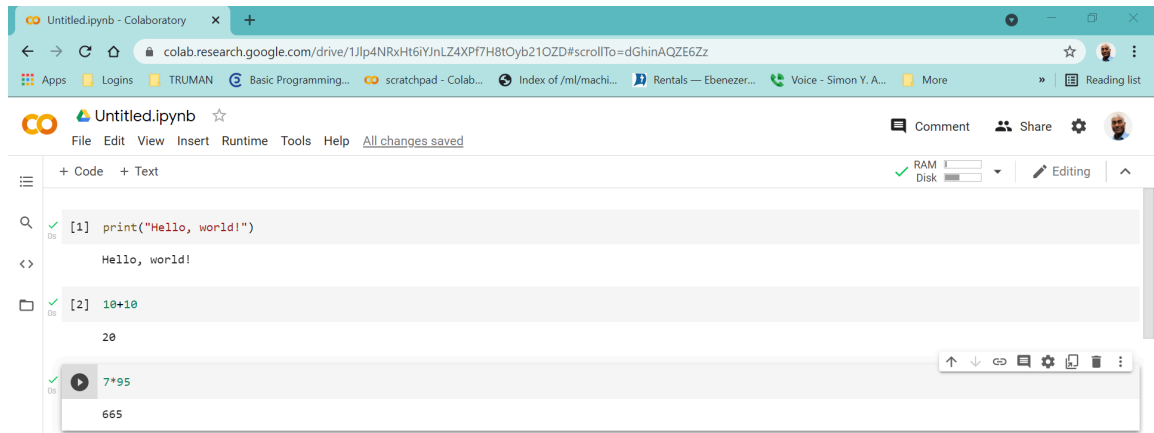
Python Arithmetic Operators

Name	Operator	Example	Result
Addition	+	5+6	11
Subtraction	-	5-6	-1
Multiplication	*	5*6	30
Division	/	5/6	0.83333
Exponentiation	**	5**2	5 ² = 25

To Write Your Third Python Code

7×95

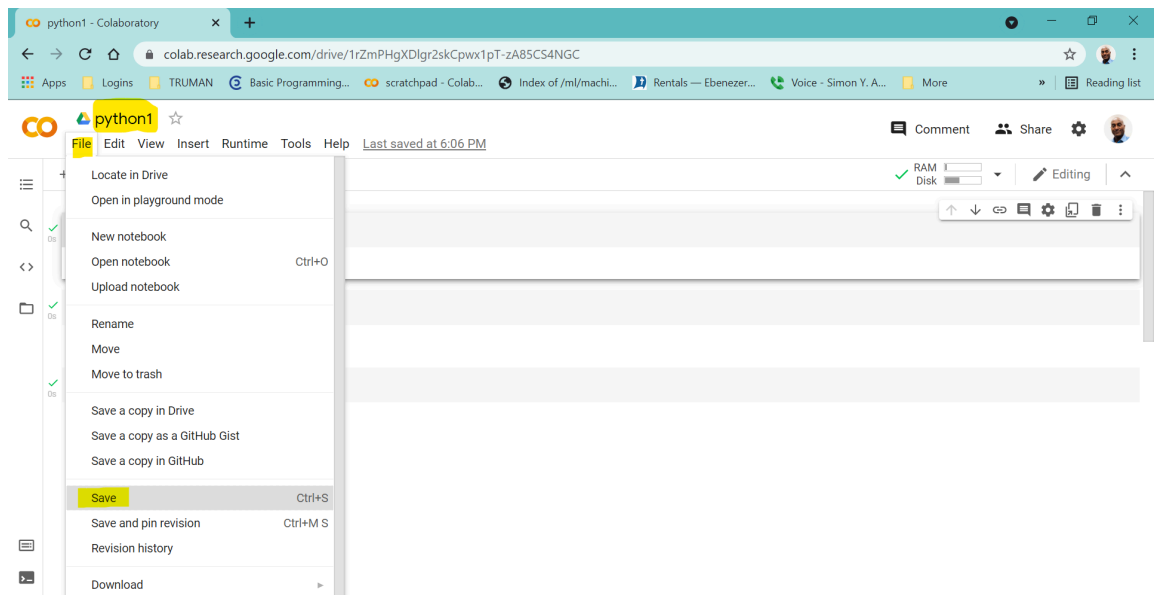
1. Click **+ Code** on a notebook to add a code cell
2. Type **7*95**
3. Press the **play** button to execute the code
4. **665** is the result



To Save Your Colab Notebook

Watch demo video

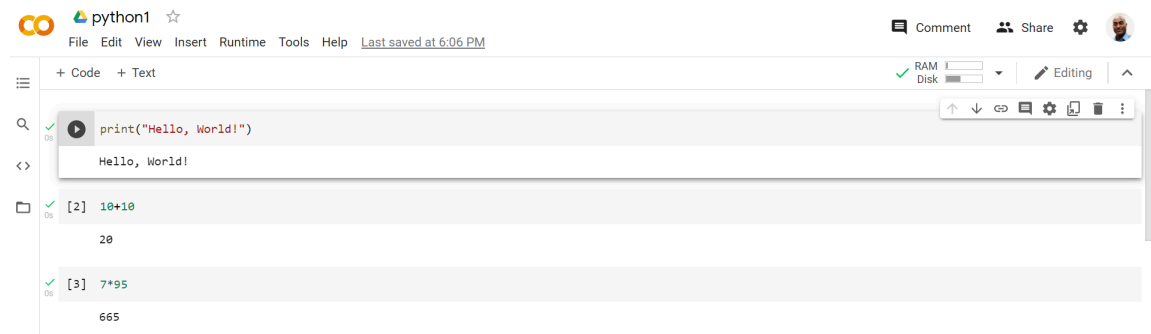
1. Click `Untitled.ipynb` at the top of the notebook
2. Delete the title and replace it by typing `python1`
3. Press the `enter` key
4. The file `python1` is now saved in your Google Drive
5. Click File and then Save
6. Close your browser



To Open Your python1 Notebook

1. Click [this link](#) or go to your [Google Drive](#).
2. Click `python1` from the list on the pop-up window
3. The notebook will open

OR click File menu then Open Notebook



General Direction: For the lab questions below:

1. Add a new code-cell on a Colab notebook
2. Type the code based on the statement of the lab
3. Execute the code

Lab 1.1.1

Add a code-cell and type `print("Hello, Python!")` and press the `play` button.

[view answer](#)

```
In [ ]: print("Hello, Python!")
```

Lab 1.1.2

Add a code-cell to your Colab notebook and write a Python code that displays the phrase "`Welcome to Python Programming Language.`"

[view answer](#)

```
In [ ]:
```

1.2 Python Arithmetic Operators

Name	Operator	Example	Result
Addition	+	5+6	11

Name	Operator	Example	Result
Subtraction	-	5-6	-1
Multiplication	*	5*6	30
Division	/	5/6	0.83333
Exponentiation	**	5**2	$5^2 = 25$

Lab 1.2.1

Add a code-cell, type `10 + 10` , and click the `play` button to the left of `10 + 10` .

[view answer](#)

In []: `10+10`

Lab 1.2.2

Write a Python code to find the value of $100 - 88$.

[view answer](#)

In []:

Lab 1.2.3

Write a Python code to perform the operation $\frac{1}{2} + 7.1 - 10 \times 3$. On a code-cell type `1/2 + 7.1 - 10*3` and run (execute) the code by pressing the Ctrl+Enter or click the `play` button.

[view answer](#)

In []:

Lab 1.2.4

Write a Python code to perform the operation $-5 + 7 - \frac{100+0.05}{10}$.

[view answer](#)

In []:

Lab 1.2.5

Write a Python code to perform the operation $\frac{1}{671000000}$.

[view answer](#)

Lab 1.2.6

For the year 2021, the Illinois state tax is 6.25%. How much state tax will one pay for a car priced at \$16,000? Write a Python code to calculate the tax.

[view answer](#)

In []:

Lab 1.2.7

Write a Python code to evaluate: $28/4 \times 2$.

[view answer](#)

In []:

[Watch demo video](#)**Lab 1.2.8**

Write a Python code to perform the operation: $7^2 + \frac{1}{\sqrt{2}}$.

[view answer](#)

In []:

Lab 1.2.9

Write a code for: $7^2 - \sqrt{8}$

[view answer](#)

In []:

Lab 1.2.10

Write a code to evaluate: $\frac{7^2 - \sqrt{8}}{25}$

[view answer](#)

In []:

1.3 Python Variables

[Watch demo video](#)

Lab 1.3.1

Given the ordered pairs $(6, 9)$ and $(5, 1)$, write a Python code to find the slope and midpoint.

[view answer](#)

In []:

Lab 1.3.2

To solve a quadratic equation of the form $ax^2 + bx + c = 0$, we can use the formula below:

$$x = \frac{-b - \sqrt{b^2 - 4ac}}{2a} \text{ or } x = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

Write a Python code to solve $2x^2 - 3x - 4$ using the quadratic formula.

[view answer](#)

In []:

Lab 1.3.3

Evaluate $\frac{1+2}{18}$ and round the result to one decimal place.

[view answer](#)

In []:

1.4 Python Comments

Lab 1.4.1

Write a Python code to find the area of a triangle with length 5 inches and width 8 inches. Use comments to explain your steps.

[view answer](#)

The Numpy Library

[Watch demo video](#)

Lab 1.5.1

Write a code to perform the operations: $|7 \times 2 - \sqrt{89}|$

[view answer](#)

In []:

Lab 1.5.2

Write a code to evaluate: $\frac{|7^2 - \sqrt{8}|}{25}$

[view answer](#)

In []:

Lab 1.5.3

The area of a triangle with two sides of a & b and an angle between the two sides C is given by the formula $A = a \times b \times \sin(C)$. Where $\sin(C)$ is the value of the sine function at C .

Find the area of a triangle with sides 10 and 12 inches and included angle of $\frac{\pi}{3}$.

[view answer](#)

In []:

Lab 1.5.4

Write a Python code to perform the operations: $\sqrt{6^2 - 4 \times 2 \times (-5)}$

[view answer](#)

1.6 input Data From Keyboard

[Watch demo video](#)**Lab 1.6.1**

Write a Python code that asks an adult user to provide weight and height. Use those values to compute the BMI. Weight in lbs and height in foot and inches.

[view answer](#)

2. Python List

[Watch demo video](#)

2.1 Creating and Accessing Lists

Lab 2.1.1

Write a Python code to create a Python **list** containing the following items:

```
1,2,3,4,5,6,7,8,9
```

- Display all items

[view answer](#)

Lab 2.1.2

Write a Python code to create a Python **list** containing the following items:

```
1,2,3,4,5,6,7,8,9
```

- Display the first item

[view answer](#)

Lab 2.1.3

Write a Python code to list the following items:

```
1,2,3,4,5,6,7,8,9
```

- Display the last item

[view answer](#)

Lab 2.1.4

Write a Python code to create a Python **list** containing the following items:

```
1,2,3,4,5,6,7,8,9
```

- Display from 2nd to 4th items.

[view answer](#)

2.2 List Operations

Lab 2.2.1

Create a Python list using the ages of US Supreme Court Judges (as of 2021):

```
72,82,66,70,66,60,53,56,49
```

- write a Python code to sort the ages

[view answer](#)**Lab 2.2.2**

Create a Python list using the ages of US Supreme Court Judges (as of 2021): 72, 82, 66, 70, 66, 60, 53, 56, 49

- write a Python code to display the oldest age

[view answer](#)**Lab 2.2.3**

Create a Python list using the ages of US Supreme Court Judges (as of 2021): 72, 82, 66, 70, 66, 60, 53, 56, 49

- write a Python code to display a list from the oldest to the youngest

[view answer](#)

3. Python Syntax

3.1 if statements

[Watch demo video](#)**Lab 3.1.1**

Write a code that checks if the quadratic equation $x^2 + 3x + 4 = 0$ has one, two or no real solutions.

[view answer](#)**Lab 3.1.2**

Write a code that checks if the quadratic equation $5x^2 + 3x - 16 = 0$ has one, two or no real solutions.

[view answer](#)

3.2 Python Functions

[Watch demo video](#)

Example: Given a, b, c , and the quadratic equation $ax^2 + bx + C = 0$. write a **function** to solve such equations. See answer in the code-cell below.

```
In [ ]: def quadra(a,b,c):
    a,b,c = float(a),float(b),float(c)    # three lines in one
    if b**2-4*a*c < 0:
        return print("No real solutions")
    elif b**2-4*a*c == 0:
        x1 = -b/(2*a)
        return print("the solution is", x1)
    else:
        x1 = (-b-(b**2-4*a*c)**.5)/(2*a)
        x2 = (-b+(b**2-4*a*c)**.5)/(2*a)
        return print("the solutions are: ",x1," and ",x2)
```

Lab 3.2.1

Use the `quadra(a,b,c)` function above to solve $-x^2 - x + 10 = 0$.

[view answer](#)

Lab 3.2.2

Use the `quadra(a,b,c)` function to solve $x^2 + x + 1 = 0$.

[view answer](#)

3.3 Syntax Errors

[Watch demo video](#)

Lab 3.3.1

Identify the error

```
b=2
c=4
b/c
```

[view answer](#)

Lab 3.3.2

Identify the error in the code below:

```
a = 1
b = 2
if a > b
    print("a is bigger than b")
```

```
elif a == b:  
    print("equal")  
else:  
    print("a is smaller than b")  
view answer
```

Lab 3.3.3

Identify the error

```
a = 1  
b = 2  
if a > b:  
    print("a is bigger than b")  
elif a = b:  
    print("equal")  
else:  
    print("a is smaller than b")
```

[view answer](#)

Lab 3.3.4

Identify the error

```
a = 1  
b = 2  
if a > b:  
    print("a is bigger than b")  
elif a == b:  
    print("equal")  
else:  
    print("a is smaller than b")
```

[view answer](#)

4. Python Loops

There are two type of loops (repeated execution of statements): `for` and `while` loops

4.1 The `for` Loop

[Watch demo video 1](#)

[Watch demo video 2](#)

Lab 4.1.1

Use the `for` loop to find the average age of the IL Supreme Court Justices given the list of ages: `x=[77, 72,73,62,77,55]` .

[view answer](#)

4.2 The `while` Loop

Example

`while` loop for Fibonacci Numbers

Fibonacci Numbers: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, â€¦

The first two terms are 1 and 1. All other terms are obtained by adding the preceding two terms.

Below is a code that lists Fibonacci numbers up to a user given count.

```
In [ ]: # type in the number of Fibonacci number you want to list and press enter.
n = input("How many Fibonacci numbers? Type a number and press enter. ")

n = int(n)           # convert it to an integer

i = 0                # set i to zero initially
a = 1                # first number is 1
b = 1                # second number is 1

while i < n:          # i starts at zero and ends at n-1
    print (a)
    i = i + 1
    c = a + b          # sum assigned to c
    a = b              # second assigned to first
    b = c              # sum assigned to second
```

How many Fibonacci numbers? Type a number and press enter. 5

1
1
2
3
5

Lab 4.2.1

List the first 20 Fibonacci numbers.

[view answer](#)

Lab 4.2.2

Write a Python code that lists all positive integers starting from 1 till a user given number.

[view answer](#)

5. Frequency Distributions and Graphs

5.1 Frequency Distributions

[Watch demo video 1](#)

[Watch demo video 2](#)

[Watch demo video 3](#)

Lab 5.1.1

Write a Python code to find the frequency counts of the following data: 20, 21, 25, 20, 29, 25, 22, 25, 21, 25

[view answer](#)

Lab 5.1.2

20 students were asked if they were registered to vote. Their responses are listed below. Construct a frequency distribution.

NO, NO, YES, YES, NO, YES, NO, NO, YES, NO, YES, NO, NO, YES, YES, YES, YES, NO, YES, YES

[view answer](#)

Lab 5.1.3

Write a Python code to construct a frequency distribution with 5 classes or bins for the following test

scores: 40,40,40,40,40,41,41,41,41,41,41,41,42,42,42,42,43,43,43,43,43,43,44,44,44

[view answer](#)

5.3 Bar Graphs and Pie Charts

[Watch demo video 1](#)

[Watch demo video 2](#)

Lab 5.3.1

The latest number of coronavirus deaths (July 2021) in Chicago by race is given below. Write a Python code to construct a bar graph.

Race	Deaths
Latinx	1859
Black	2224
White	1233
Other	309

[view answer](#)

Lab 5.3.2

The latest number of coronavirus deaths in Chicago by race is given below. Write a Python code to construct a pie chart.

Race	Deaths
Latinx	1859
Black	2224
White	1233
Other	309

[view answer](#)

5.4 Histogram, Polygon and Ogive

[Watch demo video](#)

Lab 5.4.1

A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Write a Python code to construct a histogram with 6 classes(bins).

```
46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868
```

[view answer](#)

Lab 5.4.2

Example: A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Write a Python code using `matplotlib` to construct a histogram with 6 classes(bins).

```
46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868
```

[view answer](#)

Polygon

[Watch demo video](#)

```
In [ ]: def polygon(x,n):
import matplotlib.pyplot as plt
import numpy as np

freq, bins, edges = plt.hist(x,bins=n,ec="k",range=[min(x),max(x)+1])
plt.title("Polygon")
plt.xlabel('data values')
plt.ylabel('frequency')

a1=bins[0]-(bins[1]-bins[0])
```

```

a2=bins[-1]+(bins[1]-bins[0])
bin1=np.insert(bins,[0],a1)
bin1=np.append(bin1,a2)

freq,edges,_=plt.hist(x,bins=bin1,ec='k', range=[min(x),max(x)+1])
midpoints=0.5*(edges[1:]+edges[:-1])
plt.plot(midpoints,freq)
plt.xticks(bin1)
plt.show()

```

Lab 5.4.3

A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Use the `polygon` function to construct a polygon with 6 classes(bins).

```

46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868

```

[view answer](#)

Ogive

[Watch demo video](#)

```

In [ ]: def ogive(x,n):
import matplotlib.pyplot as plt
import numpy as np

freq, bins, edges = plt.hist(x,bins=n,ec="k",range=[min(x),max(x)+1])
plt.title("Ogive")
plt.xlabel('data values')
plt.ylabel('Cumulative frequency')

a1=bins[0]-(bins[1]-bins[0])
a2=bins[-1]+(bins[1]-bins[0])
bin1=np.insert(bins,[0],a1)
bin1=np.append(bin1,a2)

freq,edges,_=plt.hist(x,bins=bin1,ec='k', range=[min(x),max(x)+1])

ogive1 = edges[:-1]
cumfreq = np.cumsum(freq)
plt.plot(ogive1,cumfreq)
plt.xticks(bin1)
plt.show()

```

Lab 5.4.4

A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Use the `ogive` function to construct an ogive with 6 classes(bins).

```
46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868
```

[view answer](#)

5.5 Dot Plot

[Watch demo video](#)

```
In [ ]: # dotplot by Patrick Fitzgerald
def dotplots(x,x_label="Data Values"):    # function name is dotplots takes a x

    import numpy as np
    import matplotlib.pyplot as plt

    data = np.array(x)    # convert to array
    values, counts = np.unique(data, return_counts=True)

    # dot plot with appropriate figure size and y-axis limits
    fig, ax = plt.subplots(figsize=(6, 2.25))
    for value, count in zip(values, counts):
        ax.plot([value]*count, list(range(count)), 'co', ms=10, linestyle='')
    for spine in ['top', 'right', 'left']:
        ax.spines[spine].set_visible(False)
    ax.yaxis.set_visible(False)
    ax.set_ylim(-1, max(counts))
    ax.set_xticks(range(min(values), max(values)+1))
    ax.tick_params(axis='x', length=0, pad=8, labelsz=12)

    plt.suptitle("Dot Plot")
    plt.ylabel("frequency")
    plt.xlabel("values(categories)")

    return plt.show()
```

Lab 5.5.1

The dataset below is the waiting time, in minutes, of customers at a fast food restaurant. Use the `dotplots` function to construct a dot plot.

```
6, 4, 4, 2, 1, 3, 5, 2, 4, 2, 5, 4, 4, 7, 4, 3, 4, 1, 2, 5
```

[view answer](#)

5.6 Stem and Leaf Plot

[Watch demo video](#)

Lab 5.6.1

Given the test scores listed below, write a Python code to construct and stem and leaf plot.

```
54, 22, 77, 92, 51, 68, 37, 27, 68, 47, 42, 38, 18, 46, 74, 77, 81, 78,  
16, 35, 77, 90, 82, 36, 50, 39, 19, 62, 70, 88, 36, 37, 53, 72, 77
```

[view answer](#)

6. Measure of Center and Variation

[Watch demo video](#)

6.1 Mean and Median

Lab 6.1.1

A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Write a Python code to compute the mean and median.

```
46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,  
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,  
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,  
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,  
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,  
103764, 105420, 109620, 146868
```

[view answer](#)

6.2 Variance and Standard Deviation

[Watch demo video](#)

Lab 6.2.1

The salaries for all the Human Resources department employees of the City of Chicago is listed below. Write a Python code to compute the **population variance and standard deviation**.

75408, 82368, 103968, 82368, 42996, 85992, 67464, 103716, 134292, 113484, 94848, 101628, 119712, 61776, 122496, 57720, 119712, 119712, 61140, 103716, 67464, 58680, 91020, 83268, 62712, 78828, 82788, 91020, 75408, 59820, 79812, 63348, 122496, 72744, 98628, 103968, 103968, 131664, 115788, 76248, 119712, 83268, 75408, 75408, 95172, 67824, 75408, 119712, 141696, 97392, 83268, 59820, 97668, 82368, 53736, 65676, 103716, 94848, 79020, 67824, 86856, 103716, 75408, 119712, 91944, 94848, 66336, 99480, 119712

[view answer](#)

Lab 6.2.2

A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Write a Python code to compute the **sample variance and standard deviation**.

46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408, 60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272, 72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484, 82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272, 88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716, 103764, 105420, 109620, 146868

[view answer](#)

Weighted Mean

[Watch demo video](#)

6.4 Grouped Mean and Standard Deviation

Lab 6.4.1

Below is a frequency table of the salaries for the Human Resources department of City of Chicago is listed below. Write a Python code to compute the **sample mean and standard deviation**.

salaries	freq
5200	9
7200	21

salaries	freq
9200	18
11200	16
13500	5

[view answer](#)

7. Measures of Position

7.1 Percentiles

[Watch demo video](#)

Lab 7.1.1

The salaries for all the Human Resources department of City of Chicago is listed below. Write a Python code to compute the 10th, 25th, 50th, 75th and 90th percentiles.

```
75408, 82368, 103968, 82368, 42996, 85992, 67464, 103716, 134292, 113484,
94848, 101628, 119712, 61776, 122496, 57720, 119712, 119712, 61140,
103716, 67464, 58680, 91020, 83268, 62712, 78828, 82788, 91020, 75408,
59820, 79812, 63348, 122496, 72744, 98628, 103968, 103968, 131664, 115788,
76248, 119712, 83268, 75408, 75408, 95172, 67824, 75408, 119712, 141696,
97392, 83268, 59820, 97668, 82368, 53736, 65676, 103716, 94848, 79020,
67824, 86856, 103716, 75408, 119712, 91944, 94848, 66336, 99480, 119712
```

[view answer](#)

7.2 SciPy and Percentile Rank

[Watch demo video](#)

Lab 7.2.1

A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Write a Python code to compute the percentile salary rank of an employee with salary 80000.

```
46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
```

72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
 82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
 88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
 103764, 105420, 109620, 146868

[view answer](#)

7.3 Box Plot

[Watch demo video](#)

Lab 7.3.1

The following data is winter temperatures in randomly selected towns and villages in Alaska (simulated data). Write a Python code to draw a box plot.

27.3	5.5	14.3	3.2	16.8	17.9,	14.2	14.2	9.6	34.7	8.9	30.1
27.7	17.7	21.8	47.7	37	24.9						

[view answer](#)

8 Discrete Probability Distributions

8.1 Expected Value and Standard Deviation

[Watch demo video](#)

Lab 8.1.1

The random variable X is defined as the number of times a graduating senior changed majors. The probability distribution of X is given below.

x	0	1	2	3	4	5	6	7	8	$p(x)$	0.197	0.212	0.255	0.182	0.09	0.04	0.02	0.003	0.001

1. What is the mean number of times students changed majors?
2. What is the standard deviation of X ?

[view answer](#)

8.2 Binomial Probability Distribution

[Watch demo video 1](#)[Watch demo video 2](#)

Lab 8.2.1

21% of all Chicagoans live in poverty. If 30 Chicagoans are randomly selected, find the probability that

1. Exactly 1 of them live in poverty
2. At most 4 of them live in poverty
3. At least 4 of them live in poverty

[view answer](#)

Binomial Probability Plot

Example

Use the `binomplot` function below to plot a binomial distribution with $n = 20$ and probability of success is 0.70.

```
In [ ]: # binomial graph with n and p
def binomplot(n,p):
    from scipy import stats
    import numpy as np
    import matplotlib.pyplot as plt

    x = np.arange(0,n+1)
    y = stats.binom.pmf(x,n,p)
    plt.bar(x,y,width=0.2)

    plt.suptitle("Binomial Plot")
    plt.xlabel("x values")
    plt.ylabel("probability")

    return plt.show()
```

Lab 8.2.2

Use the `binomplot` function to plot a binomial distribution with $n = 10$ and probability of success is 0.60.

[view answer](#)

9. Continuous Probability Distribution

9.1 Uniform Probability Distribution

Lab 9.1.1

Example: The amount of time, in minutes, that a person waits for a train, in a train station, is uniformly distributed between 5 and 15 minutes, inclusive.

- What is the probability that a person waits fewer than 15 minutes?

[view answer](#)

9.2 Normal Probability Distribution

[Watch demo video](#)

Lab 9.2.1

In a standard normal distribution, where the mean is zero and standard deviation is one, what is the probability of obtaining a value (z-value) less than 1.5?

[view answer](#)

[Watch demo video](#)

Lab 9.2.2

The average college cost per year is \$27191 with the standard deviation of \$7126 . Assume the distribution is normal.

- Find the probability that a randomly selected college will cost less than \$27726 per year.

[view answer](#)

Inverse Normal

[Watch demo video](#)

Lab 9.2.3

The average college cost per year is 27,191 with standard deviation of 7,126. Assume the distribution is normal.

- Find the 75th percentile for this distribution

[view answer](#)

9.3 Central Limit Theorem: mean

[Watch demo video](#)

Lab 9.3.1

The lengths of metal screws are normally distributed, with a mean of 5 inches, and standard deviation of 1 inch.

1. If 10 screws are chosen at random, what is the probability that their mean length is less than 6 inches?
2. If 10 screws are chosen at random, what is the probability that their mean length is between 5 and 6 inches?

[view answer](#)

Confidence Intervals

10.1 Confidence Intervals for Mean (z distribution)

[Watch demo video 1](#)

[Watch demo video 2](#)

Lab 10.1.1

Exam scores are normally distributed with a standard deviation of 15 points. The population mean is unknown. A random sample of 50 scores has a mean of 80. Construct a 90% confidence interval for the population mean. Use the `conf_z_mean` function above.

[view answer](#)

10.2 Confidence Intervals for Mean (t distribution)

[Watch demo video](#)

Lab 10.2.1

A random sample of 25 test scores was selected from an approximately normally distributed population of test scores. The mean and standard deviation are 85 and 7 respectively.

1. Compute the margin of error using the `error_t_mean` function above for constructing a 90% confidence interval.
2. Construct a 90% confidence interval using the `conf_t_mean` function above.

[view answer](#)

10.3 Confidence Intervals for Proportions (z distribution)

[Watch demo video](#)

Lab 10.3.1

In a random sample of 120 adults, 51 are fully vaccinated for Covid19. Construct a 90% confidence interval using the `conf_prop` function above.

[view answer](#)

Lab 10.3.2

In a random sample of 120 adults, 50% are fully vaccinated for Covid19. Construct a 95% confidence interval using the `conf_prop` function above.

[view answer](#)

Lab 10.3.3

Town officials want to determine the current percentage of residents vaccinated for Covid19. How many residents should be surveyed in order to be 90% confident that the estimated proportion is within five percentage points of the true population proportion who are vaccinated for Covid19? Previously the proportion was 0.55. Use the `samp_pro` function below.

[view answer](#)

11. Hypothesis Testing

11.1 Hypothesis Testing: Mean, z-test

[Watch demo video](#)

Lab 11.1.1

A government official claims that the average recovery time for Covid19 patients with mild symptoms is less than 2 weeks. A random sample of 25 patients with mild symptoms has a mean recovery time of 2.4 weeks with standard deviation of 0.9 weeks. Assume recovery time is normally distributed with standard deviation of 0.5 (σ) weeks. Test the government's official claim at α of 0.05. Use the `z_test_mean` function above.

[view answer](#)

11.2 Hypothesis Testing: Mean, t-test

[Watch demo video](#)

Lab 11.2.1

A government official claims that the average recovery time for Covid19 patients with mild symptoms is 1.5 weeks. A random sample of 25 patients with mild symptoms. Their recovery time is given below. Assume recovery time is normally distributed with standard deviation of 0.5 (σ) weeks. Test the government's official claim at α of 0.05. Use the `z_test_mean` function above.

2.5, 3.0, 1.5, 2.4, 2.6, 2.1, 1.8, 1.5, 1.7, 1.9, 3.2, 2.3, 2.0, 1.8,
1.2, 1.5, 2.3, 2.5, 1.8, 2.2, 3.3, 2.3, 1.6, 2.5, 2.2

[view answer](#)

12.3 Hypothesis Testing: Proportions, z-test

[Watch demo video](#)

Lab 12.3.1

The President of USA believes that more than 50% of the US population are fully vaccinated for Covid19. In a survey of 1000 residents, 510 were fully vaccinated. Test the president's claim at α level of 0.05. Use the `z_test_prop` function.

[view answer](#)

12. Linear Regression and Correlation

12.1 Scatter Plot and Correlation

[Demo Video Link](#) Scatter Plot

[Watch demo video 2](#) Correlation Coefficient

Lab 12.1.1

The number of years (age) x and prices y , for 10 cars is listed below. Compute the correlation coefficient and display the scatter plot.

x	1	0	3	5	14	7	4	16	9	10
y	17000	20000	15000	10000	5000	9000	10000	2000	8000	9000

[view answer](#)

12.3 Linear Regression

[Watch demo video](#)

Lab 12.3.1

The number of years (age) x and prices y , for 10 cars is listed below. Compute the correlation coefficient (r -value), estimated slope and y -intercept of the regression line and the p -value for testing the significance of correlation.

x	1	0	3	5	14	7	4	16	9	10
y	17000	20000	15000	10000	5000	9000	10000	2000	8000	9000

[view answer](#)

13. Analysis of Categorical Data

13.1 Goodness of Fit

[Watch demo video](#)

Lab 13.1.1

The table below shows the distribution of the 2020 US presidential electoral votes. An official claims that the distribution is proportional with the popular vote, with 51.3% for Biden and 46.9% for Trump. Test the official's claim at $\alpha = 0.10$.

Candidate	Electoral-Votes	Popular vote
Biden	306	51.3%
Trump	232	46.9%

[view answer](#)

13.2 Test of Independence

[Watch demo video](#)

Lab 13.2.1

The contingency table below shows the prison population of Illinois in June 2021. Test the hypothesis that race (Black, White and Hispanic) and crime-class are independent at 5% level of significance.

Class|Black|Hispanic|White ---|---|---|---| 1|1734|475|1281 2|2071|373|1938 3|678|103|935
4|534|92|560| X|5644|1661|2613 Murder|4094|927|1277

Use the `chi2_test_of_indp` function above.

[view answer](#)

14. Advanced Statistics Topics

14.1 Descriptive Statistics with Pandas

[Watch demo video](#)

Pandas Library

Lab 14.1.1

Write a Python code to convert the dataset below to pandas data frame with variable name `gender` .

```
x = ['male', 'female', 'female', 'male', 'male']
```

[view answer](#)

Frequency and Relative Frequency with Pandas

Lab 14.1.2

Write a Python code using pandas to find the frequency counts of the following data: 20, 21, 25, 20, 29, 25, 22, 25, 21, 25

[view answer](#)

Lab 14.1.3

Write a Python code using pandas to find the *relative* frequency counts of the following data: 20, 21, 25, 20, 29, 25, 22, 25, 21, 25

[view answer](#)

Lab 14.1.4

20 students were asked if they were registered to vote. Their responses are listed below. Construct a frequency distribution using pandas.

NO, NO, YES, YES, NO, YES, NO, NO, YES, NO, YES, NO, NO, YES, YES, YES, YES, NO, YES, YES

[view answer](#)

Summary Statistics

Lab 14.1.5

A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Write a Python code using pandas to compute summary statistics: mean, standard deviation, minimum, 1st quartile, 2nd quartile, 3rd quartile and maximum salary.

46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408, 60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272, 72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484, 82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272, 88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716, 103764, 105420, 109620, 146868

[view answer](#)

Skewness and Kurtosis

Lab 14.1.6

A random sample of 54 annual salaries of Aviation employees of the City of Chicago is listed below. Write a Python code with `pandas` to compute the skewness and kurtosis values.

```
46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868
```

[view answer](#)

14.2 Uploading Dataset

[Watch demo video](#) Read CSV from URL

[Watch demo video](#) Upload Excel from Computer

Lab 14.2.1

Write a Python code to read a dataset in csv format from a website (URL) into Google Colab. Use the URL below. Convert the dataset to pandas dataframe and print basic information about the data set using `x_df.info()` method.

<https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data.csv>

[view answer](#)

14.3 Descriptive Statistics (multiple columns)

[Watch demo video](#)

Lab 14.3.1

Write a Python code to read the dataset with the url below and find the mean, median, standard deviation, minimum, maximum, 25th and 75th percentiles (descriptive statistics) of all columns in the dataset.

<https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data.csv>

[view answer](#)

Lab 14.3.2

Write a Python code to read the dataset with url below and find the mean, std, and size of the `actual_weight` grouped by `country_of_origin` .

<https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data.csv>

[view answer](#)

Lab 14.3.3

Write a Python code to find the mean, std, size of the `actual_weight` grouped by `country_of_origin` and `route` .

<https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data.csv>

[view answer](#)

Lab 14.3.4

Write a Python code to display a boxplot of the `actual_weight` grouped by `country_of_origin` .

<https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data.csv>

[view answer](#)

14.4 Visualization

Lab 14.4.1

The dataset below (URL) contains students test1_score, test2_score, test3_score, gender, level, section etc.

Write a Python code to display a frequency bar graph for `section` .

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

[view answer](#)

Lab 14.4.2

The dataset below (URL) contains students test1_score, test2_score, test3_score, gender, level, section etc.

Write a Python code to display a frequency pie chart for `section` .

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

[view answer](#)

14.5 Regression

Multiple Linear Regression

[Watch demo video](#)

Lab 14.5.1

The dataset in the website (URL below) includes 20 columns about home sale, including price, number of bedrooms and bathrooms, areas etc. The data includes 21,613 entries.

1. Write a Python code to find the least-squares prediction equation,
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \epsilon$$
 for these data where, y is `price`, x_1 is `bedrooms`, x_2 `sqft_basement` and x_3 is `sqft_living`.
2. Use the overall F test to determine whether the model contributes significant information to the prediction of y with 1% level of significance.
3. What is the value of R-squared. That is, the coefficient of determination.
4. Does the `sqft_living`, x_3 , contribute significant information for the prediction of the price, y , given that x_1 and x_2 are already in the model? (use $\alpha = 0.05$)

https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/kc_house_data.csv

[view answer](#)

Logistic Regression

Lab 14.5.2

The data set below (URL) contains the occurrence of coronary heart disease, chd, tobacco, obesity, age, alcohol, etc. For chd, 0 means patient has no chd and 1 means patient has chd.

1. Write a Python code to read the data set and display the first five rows.
2. Write a Python code to fit a logistic regression model where the dependent variable is `chd` and the predictor variables are `ldl`, `sbs` and `age`. Use 5% level of significance.

<https://raw.githubusercontent.com/simeonaman/demo/master/heart.csv>

[view answer](#)

14.6 ANOVA

[Watch demo video](#)

One-way ANOVA

Lab 14.6.1

The data set below (URL) contains the students test1_score, gender, level, section etc.

1. Write a Python code to read the data set and display the first and last five rows.
2. Write a Python code to find `test3_score` means, std and size for each level
3. Write a Python code to perform ANOVA to test if there is sufficient evidence to indicate a difference in the averages of `test1_scores` among the three `levels` (low, medium, high). Use 10% level of significance.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

[view answer](#)

Two-way ANOVA (Block Design)

Lab 14.6.2

The data set below (URL) contains students test1_score, gender, level, section etc.

1. Write a Python code to read the dataset and display the first five rows.
2. Write a program to find `test3_score` means grouped by `level` and `gender`.

3. Write a Python code to perform ANOVA block design to test if there is sufficient evidence to indicate a difference in the averages of `test3_score` among the three levels (low, medium, high) using gender as blocks. Use 10% level of significance.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

[view answer](#)

Two-Factor ANOVA (Interactions)

Lab 14.6.3

The dataset below (URL) contains students `test1_score` , `gender` , `level` , `section` etc.

1. Write a Python code to read the dataset and display the first 5 rows.
2. Write a Python code to display the means grouped by `level` and `interaction` for `test3_score` .
3. Write a Python code to perform a two-factor ANOVA with `level` and `section` . Do the data provide sufficient evidence to indicate an interaction between `level` and `section` ? Use 10% level of significance.

https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv

[view answer](#)

Lab Answers

Lab 1.1.1

```
In [ ]: print("Hello, World")
```

Hello, World

Lab 1.1.2

```
In [ ]: print('Welcome to Python Programming Language.')
```

Welcome to Python Programming Language.

Lab 1.2.1

```
In [ ]: 10+10
```

Out[]: 20

Lab 1.2.2

```
In [ ]: 100-88
```

Out[]: 12

Lab 1.2.3

```
In [ ]: 1/2+7.1-10*3
```

Out[]: -22.4

Lab 1.2.4

```
In [ ]: -5 + 7 - (100+.05)/10
```

Out[]: -8.004999999999999

Lab 1.2.5

In []: `1/671000000`

Out[]: `1.4903129657228019e-09`

Lab 1.2.6

In []: `6.25*100*16000`

Out[]: `10000000.0`

Lab 1.2.7

In []: `28/4*2`

Out[]: `14.0`

Lab 1.2.8

In []: `7**2+1/2**(1/2)`

Out[]: `49.707106781186546`

Lab 1.2.9

In []: `7**2-8**(1/2)`

Out[]: `46.17157287525381`

Lab 1.2.10

In []: `(7**2-8**(1/2))/25`

Out[]: `1.8468629150101523`

Lab 1.3.1

In []: `x1 = 6`
`y1 = 9`
`x2 = 5`
`y2 = 1`

```

m = (y2-y1)/(x2-x1)

x = (x1+x2)/2
y = (y1+y2)/2

print('slope is',m)
print('mid point is (',x,',',y,')')

```

slope is 8.0
mid point is (5.5 , 5.0)

Lab 1.3.2

```

In [ ]: a = 2
        b = -3
        c = -4

        x1 = -b-(b**2-4*a*c)**(1/2)
        x1 = x1/(2*a)

        x2 = -b+(b**2-4*a*c)**(1/2)
        x2 = x2/(2*a)

        print("one solution is ", x1)
        print("another solution is ", x2)

```

one solution is -0.8507810593582121
another solution is 2.350781059358212

Lab 1.3.3

```

In [ ]: round((1+2)/18,1)

```

Out[]: 0.2

OR

```

In [ ]: a = (1+2)/18
        round(a,1)

```

Out[]: 0.2

Lab 1.4.1

```

In [ ]: length = 5
        width = 8

```

```
area = length*width # the area of a rectangle is length times width
print(area)         # display the result
```

40

Lab 1.5.1

```
In [ ]: abs(7*2-89**(1/2))
```

```
Out[ ]: 4.566018867943397
```

OR

```
In [ ]: a = 7*2-89**(1/2)
abs(a)
```

```
Out[ ]: 4.566018867943397
```

Lab 1.5.2

```
In [ ]: (abs(7**2-8**(1/2)))/25
```

```
Out[ ]: 1.8468629150101523
```

OR

```
In [ ]: a = 7**2-8**(1/2)
b = abs(a)
b/25
```

```
Out[ ]: 1.8468629150101523
```

Lab 1.5.3

```
In [ ]: import numpy as np

sin = np.sin(np.pi/3)

Area = 10*12*sin

print(Area)
```

103.92304845413263

Lab 1.5.4

```
In [ ]: import numpy as np

np.sqrt(6**2-4*2*(-5))
```

```
Out[ ]: 8.717797887081348
```

OR

```
In [ ]: import numpy as np

a = 6**2-4*2*(-5)
np.sqrt(a)
```

```
Out[ ]: 8.717797887081348
```

Lab 1.6.1

```
In [ ]: w=input("Enter weight in pounds:") #ask the user for a number
h1 = input("Enter height feet only:")
h2 = input("Enter inches:")
w,h1,h2 = float(w),float(h1),float(h2) # convert the number to float
bmi = 703*(w/(h1*12+h2)**2)
print("The BMI is", bmi)
```

```
Enter weight in pounds:180
Enter height feet only:5
Enter inches:9
The BMI is 26.578449905482042
```

Lab 2.1.1

```
In [ ]: x = [1,2,3,4,5,6,7,8,9]
x
```

```
Out[ ]: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Lab 2.1.2

```
In [ ]: x = [1,2,3,4,5,6,7,8,9]
x[0]
```

```
Out[ ]: 1
```

Lab 2.1.3

```
In [ ]: x = [1,2,3,4,5,6,7,8,9]
        x[-1]
```

```
Out[ ]: 9
```

Lab 2.1.4

```
In [ ]: x = [1,2,3,4,5,6,7,8,9]
        x[1:4]
```

```
Out[ ]: [2, 3, 4]
```

Lab 2.2.1

```
In [ ]: x = [72,82,66,70,66,60,53,56,49]

        x.sort()
        print(x)
```

```
[49, 53, 56, 60, 66, 66, 70, 72, 82]
```

Lab 2.2.2

```
In [ ]: x = [72,82,66,70,66,60,53,56,49]

        max(x)
```

```
Out[ ]: 82
```

Lab 2.2.3

```
In [ ]: x = [72,82,66,70,66,60,53,56,49]

        x.sort()
        x.reverse()
```

```
In [ ]: print(x)
```

```
[82, 72, 70, 66, 66, 60, 56, 53, 49]
```

Lab 3.1.1

```
In [ ]: a = 1  # coefficient of x^2
        b = 3  # coefficient of x
        c = 4  # constant term

        d = b**2 - 4*a*c

        if d < 0:
            print("No real solution")
        elif d == 0:
            print("One real solution")
        else:
            print("Two real solutions")
```

No real solution

Lab 3.1.2

```
In [ ]: a = 5  # coefficient of x^2
        b = 3  # coefficient of x
        c = -16 # constant term

        d = b**2 - 4*a*c

        if d < 0:
            print("No real solution")
        elif d == 0:
            print("One real solution")
        else:
            print("Two real solutions")
```

Two real solutions

Lab 3.2.1

```
In [ ]: def quadra(a,b,c):
        a,b,c = float(a),float(b),float(c)
        if b**2-4*a*c < 0:
            return print("No real solutions")
        elif b**2-4*a*c == 0:
            x1 = -b/(2*a)
            return print("the solution is", x1)
        else:
            x1 = (-b-(b**2-4*a*c)**.5)/(2*a)
            x2 = (-b+(b**2-4*a*c)**.5)/(2*a)
            return print("the solutions are: ",x1," and ",x2)
```

```
In [ ]: quadra(-1,-1,10) # in this case a=-1 b=-1 c=10
```

the solutions are: 2.7015621187164243 and -3.7015621187164243

Lab 3.2.2

```
In [ ]: def quadra(a,b,c):
        a,b,c = float(a),float(b),float(c)
        if b**2-4*a*c < 0:
            return print("No real solutions")
        elif b**2-4*a*c == 0:
            x1 = -b/(2*a)
            return print("the solution is", x1)
        else:
            x1 = (-b-(b**2-4*a*c)**.5)/(2*a)
            x2 = (-b+(b**2-4*a*c)**.5)/(2*a)
            return print("the solutions are: ",x1," and ",x2)
```

```
In [ ]: quadra(1,1,1) # heare a=1 b=1 c=1
```

No real solutions

Lab 3.3.1

```
In [ ]: b=2
        c=4
        b/c
```

```
File "<ipython-input-86-158994d5983a>", line 2
    c=4
    ^
```

IndentationError: unexpected indent

Lab 3.3.2 `if a > b:`

```
In [ ]: a = 1
        b = 2
        if a > b
            print("a is bigger than b")
        elif a = b:
            print("equal")
        else:
            print("a is smaller than b")
```

```
File "<ipython-input-87-871e93d2a39b>", line 3
    if a > b
        ^
```

SyntaxError: invalid syntax

Lab 3.3.3

```
In [ ]: a = 1
        b = 2
```

```
if a > b:
    print("a is bigger than b")
elif a = b:
    print("equal")
else:
    print("a is smaller than b"):
```

File "<ipython-input-88-128f45613308>", line 5

```
    elif a = b:
```

^

SyntaxError: invalid syntax

Lab 3.3.4

```
In [ ]: a = 1
        b = 2
        if a > b:
            print("a is bigger than b")
        elif a == b:
            print("equal")
        else:
            print("a is smaller than b")
```

File "<ipython-input-90-9aa3a9224cf2>", line 6

```
    print("equal")
```

^

SyntaxError: EOL while scanning string literal

Lab 4.1.1

```
In [ ]: age = [77, 72,73,62,77,55]

        sum = 0

        for i in age:
            sum = sum + i

        aver = sum/len(age)

        print("The average age of IL Supreme Court Justices is ", aver)
```

The average age of IL Supreme Court Justices is 69.33333333333333

Lab 4.2.1

```
In [ ]: n = 20
```



```

i = 0
a = 1
b = 1

while i < n:
    print (a)
    i = i + 1
    c = a + b
    a = b
    b = c

```

```

1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
1597
2584
4181
6765

```

Lab 4.2.2

```

In [ ]: n = input("Type in a maximum positive integer and press enter" )
        n = int(n)

        i = 1

        while i < n+1:
            print(i)
            i = i+1

```

```

Type in a maximum positive integer and press enter5
1
2
3
4
5

```

Lab 4.2.3

```
In [ ]: n = input("Type in a positive integer and press enter ")
        n = int(n)

        if n == 0 or n == 1:
            f = 1
        elif n < 0:
            print("Error: non-negative integers only")
        else:
            i = 1
            f = 1
            while i < n+1:
                f = f*i
                i = i + 1
            print("n factorial is",f)
```

Type in a positive integer and press enter 10
n factorial is 3628800

Lab 5.1.1

```
In [ ]: import numpy as np

        x = [20, 21, 25, 20, 29, 25, 22, 25, 21, 25]

        np.unique(x,return_counts=True)
```

Out[]: (array([20, 21, 22, 25, 29]), array([2, 2, 1, 4, 1]))

Lab 5.1.2

```
In [ ]: import numpy as np

        x = ['NO', 'NO', 'YES', 'YES', 'NO', 'YES', 'NO', 'NO', 'YES', 'NO', 'YES',
              'NO', 'NO', 'YES', 'YES', 'YES', 'YES', 'NO', 'YES', 'YES']
        np.unique(x,return_counts=True)
```

Out[]: (array(['NO', 'YES'], dtype='<U3'), array([9, 11]))

Lab 5.1.3

```
In [ ]: import numpy as np

        x = [40,40,40,40,40,41,41,41,41,41,41,42,42,42,42,43,43,43,43,43,43,44,44,44,
              44,45]

        freq,bin_edges = np.histogram(x,5,density=False)
```

```
print(bin_edges)
print(freq)
```

```
[40. 41. 42. 43. 44. 45.]
[5 6 4 6 5]
```

40 to 41 has 5 scores 41 to 42 has 6 scores 42 to 43 has 4 scores 43 to 44 has 6 scores 44 to 45 has 5 scores

scores	freq
40-41	5
41-42	6
42-43	4
43-44	6
44-45	5

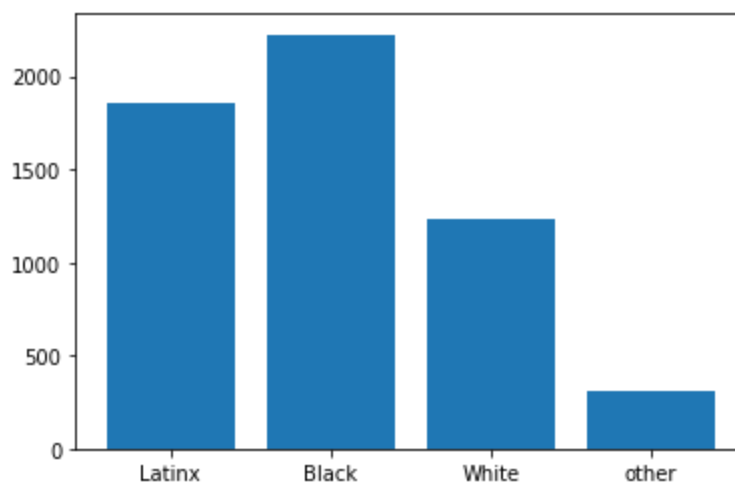
Lab 5.3.1

```
In [ ]: import matplotlib.pyplot as plt

race = ['Latinx', 'Black', 'White', 'other']
death = [1859, 2224, 1233, 309]

plt.bar(race, death)
```

```
Out[ ]: <BarContainer object of 4 artists>
```



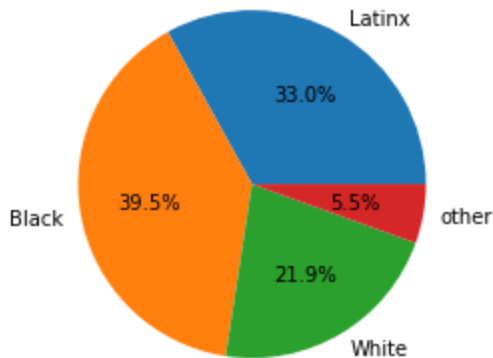
5.3.2

```
In [ ]: import matplotlib.pyplot as plt

race = ['Latinx', 'Black', 'White', 'other']
death = [1859, 2224, 1233, 309]

plt.pie(death, labels=race, autopct='%1.1f%%')
```

```
Out[ ]: ([<matplotlib.patches.Wedge at 0x7fdd012ef410>,
<matplotlib.patches.Wedge at 0x7fdd012efb50>,
<matplotlib.patches.Wedge at 0x7fdd012f9410>,
<matplotlib.patches.Wedge at 0x7fdd012f9d10>],
[Text(0.5584906875918265, 0.9476751299222794, 'Latinx'),
Text(-1.0828050332538752, -0.19373502512471624, 'Black'),
Text(0.5627194086838145, -0.9451702847056388, 'White'),
Text(1.0836598543140907, -0.18889499767850831, 'other')],
[Text(0.3046312841409962, 0.5169137072303341, '33.0%'),
Text(-0.5906209272293864, -0.10567365006802702, '39.5%'),
Text(0.3069378592820806, -0.5155474280212574, '21.9%'),
Text(0.5910871932622312, -0.10303363509736815, '5.5%')])
```



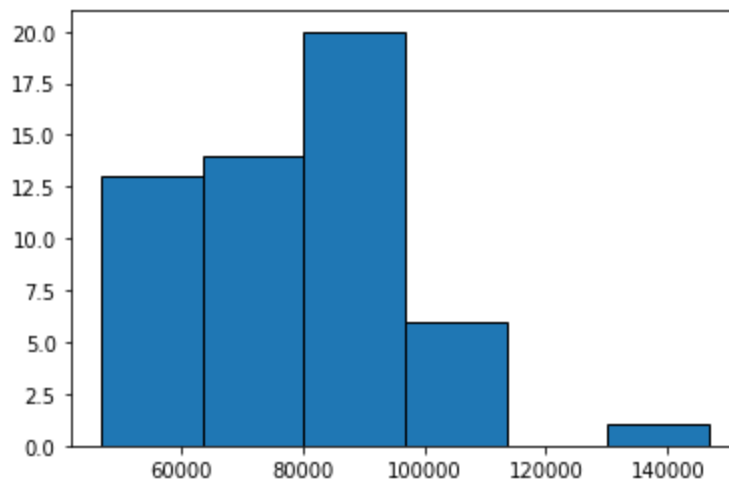
Lab 5.4.1

```
In [ ]: import matplotlib.pyplot as plt

x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868]

plt.hist(x, bins=6, ec='k')
```

```
Out[ ]: (array([13., 14., 20., 6., 0., 1.]),
array([ 46776.,  63458.,  80140.,  96822., 113504., 130186., 146868.]),
<a list of 6 Patch objects>)
```



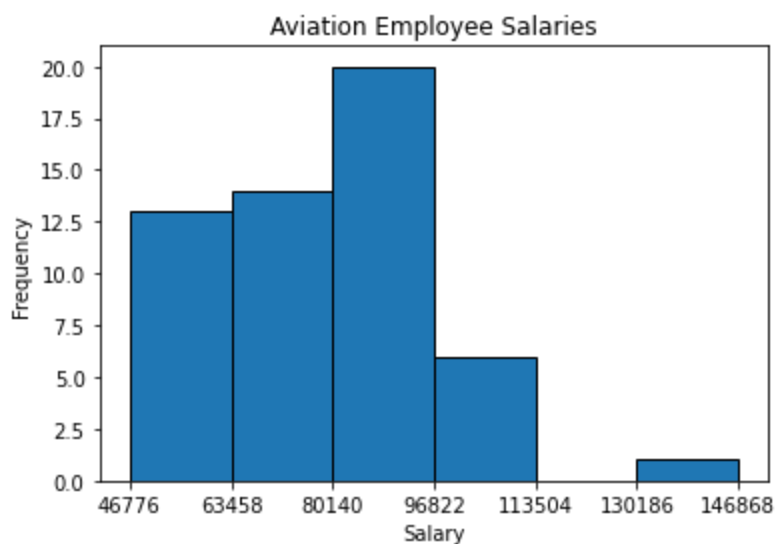
Lab 5.4.2

```
In [ ]: import matplotlib.pyplot as plt

x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
     60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
     72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
     82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
     88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
     103764, 105420, 109620, 146868]

freq,bins,_ = plt.hist(x,bins=6,ec='k')
plt.xticks(bins)
plt.title("Aviation Employee Salaries")
plt.xlabel("Salary")
plt.ylabel("Frequency")
```

```
Out[ ]: Text(0, 0.5, 'Frequency')
```



Lab 5.4.3

Copy, paste and run the `polygon` function below. Then write a Python code to call the function with the given data.

```
In [ ]: def polygon(x,n):
import matplotlib.pyplot as plt
import numpy as np

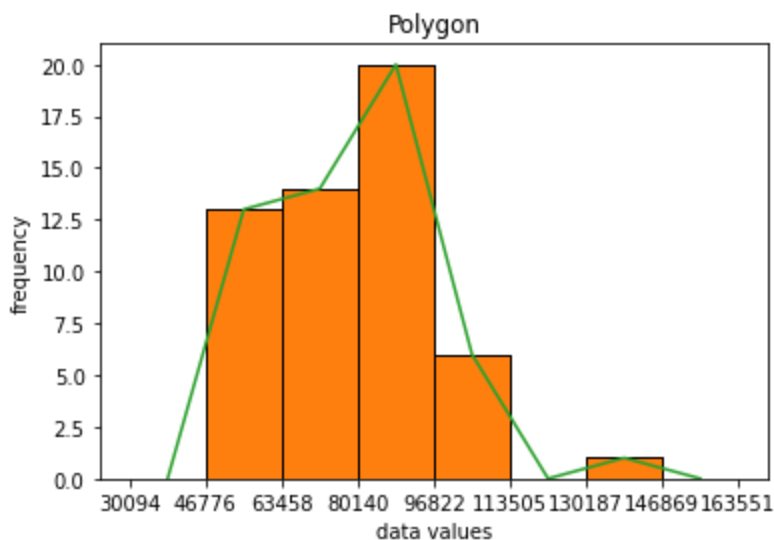
freq, bins, edges = plt.hist(x,bins=n,ec="k",range=[min(x),max(x)+1])
plt.title("Polygon")
plt.xlabel('data values')
plt.ylabel('frequency')

a1=bins[0]-(bins[1]-bins[0])
a2=bins[-1]+(bins[1]-bins[0])
bin1=np.insert(bins,[0],a1)
bin1=np.append(bin1,a2)

freq,edges,_=plt.hist(x,bins=bin1,ec='k', range=[min(x),max(x)+1])
midpoints=0.5*(edges[1:]+edges[:-1])
plt.plot(midpoints,freq)
plt.xticks(bin1)
plt.show()

In [ ]: x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868]

polygon(x,6)
```



Lab 5.4.4

Copy the 'ogive' function, paste and run it below. Then use it to make an ogive.

```
In [ ]: def ogive(x,n):
import matplotlib.pyplot as plt
import numpy as np

freq, bins, edges = plt.hist(x,bins=n,ec="k",range=[min(x),max(x)+1])
plt.title("Ogive")
plt.xlabel('data values')
plt.ylabel('Cumulative frequency')

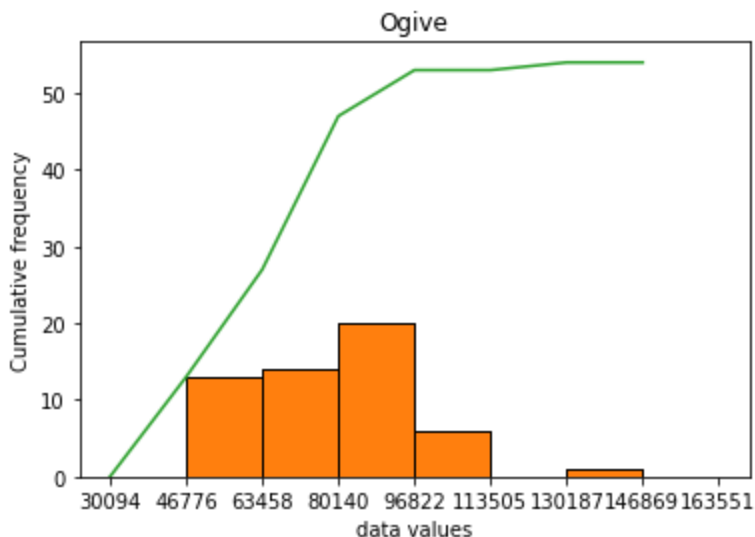
a1=bins[0]-(bins[1]-bins[0])
a2=bins[-1]+(bins[1]-bins[0])
bin1=np.insert(bins,[0],a1)
bin1=np.append(bin1,a2)

freq,edges,_,=plt.hist(x,bins=bin1,ec='k', range=[min(x),max(x)+1])

ogive1 = edges[:-1]
cumfreq = np.cumsum(freq)
plt.plot(ogive1,cumfreq)
plt.xticks(bin1)
plt.show()

In [ ]: x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868]

ogive(x,6)
```



Lab 5.5.1

Copy the `dotplots` function and paste it below. Then use it to draw a dot plot.

```
In [ ]: # dotplot by Patrick Fitzgerald
def dotplots(x,x_label="Data Values"): # function name is dotplots takes a x

    import numpy as np
    import matplotlib.pyplot as plt

    data = np.array(x) # convert to array
    values, counts = np.unique(data, return_counts=True)

    # dot plot with appropriate figure size and y-axis limits
    fig, ax = plt.subplots(figsize=(6, 2.25))
    for value, count in zip(values, counts):
        ax.plot([value]*count, list(range(count)), 'co', ms=10, linestyle='')
    for spine in ['top', 'right', 'left']:
        ax.spines[spine].set_visible(False)
    ax.yaxis.set_visible(False)
    ax.set_ylim(-1, max(counts))
    ax.set_xticks(range(min(values), max(values)+1))
    ax.tick_params(axis='x', length=0, pad=8, labelsz=12)

    plt.suptitle("Dot Plot")
    plt.ylabel("frequency")
    plt.xlabel(x_label)

    return plt.show()
```

```
In [ ]: x = [6, 4, 4, 2, 1, 3, 5, 2, 4, 2, 5, 4, 4, 7, 4, 3, 4, 1, 2, 5]
dotplots(x, "Waiting Time")
```

**Lab 5.6.1**

First install stemgraphic

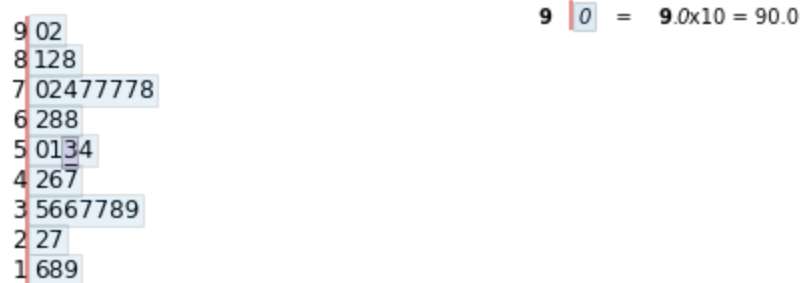
```
In [ ]: pip install stemgraphic
```



```
In [ ]: import stemgraphic

x = [54, 22, 77, 92, 51, 68, 37, 27, 68, 47, 42, 38, 18, 46, 74, 77, 81, 78, 16,
      35, 77, 90, 82, 36, 50, 39, 19, 62, 70, 88, 36, 37, 53, 72, 77]
stemgraphic.stem_graphic(x,scale=10,aggregation=False)
```

```
Out[ ]: (<Figure size 540x216 with 1 Axes>,
        <matplotlib.axes._axes.Axes at 0x7fefe45b7410>)
```



Lab 6.1.1

```
In [ ]: import numpy as np

x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
      60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
      72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
      82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
      88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
      103764, 105420, 109620, 146868]

print("mean and median")
print(np.mean(x))
print(np.median(x))
```

```
mean and median
78460.0
78366.0
```

Interpretation: The mean and median are 78460.0 and 78366.0 respectively.

Lab 6.2.1

```
In [ ]: import numpy as np

x=[75408, 82368, 103968, 82368, 42996, 85992, 67464, 103716, 134292, 113484,
    94848, 101628, 119712, 61776, 122496, 57720, 119712, 119712, 61140, 103716,
```

```
67464, 58680, 91020, 83268, 62712, 78828, 82788, 91020, 75408, 59820, 79812,
63348, 122496, 72744, 98628, 103968, 103968, 131664, 115788, 76248, 119712,
83268, 75408, 75408, 95172, 67824, 75408, 119712, 141696, 97392, 83268,
59820, 97668, 82368, 53736, 65676, 103716, 94848, 79020, 67824, 86856,
103716, 75408, 119712, 91944, 94848, 66336, 99480, 119712]
```

```
print('the population variance and standared devaition are')
print(np.var(x,ddof=0))
print(np.std(x,ddof=0))
```

the population variance and standared devaition are
490101843.4177693
22138.24390998006

Lab 6.2.2

In []: `import numpy as np`

```
x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868]
```

```
print('The sample variance and standard deviation')
print(np.var(x,ddof=1))
print(np.std(x,ddof=1))
```

The sample variance and standard deviation
333192547.0188679
18253.562584297564

Interpretation: The sample variance and standard deviation are 333192547.0188679 and 18253.562584297564 respectively.

Lab 6.4.3

In []: `import numpy as np`

```
x = [5200, 7200, 9200, 11200, 13500]
f = [9,21,18,16,5]

np.average(x,weights=f)
```

Out[]: 8844.927536231884

Interpretaton: The sample mean is 8844.927536231884

```
In [ ]: import numpy as np

x = [5200, 7200, 9200, 11200, 13500]
f = [9,21,18,16,5]

np.sqrt(np.cov(x,fweights=f,ddof=1))
```

Out[]: 2350.6905263250064

Interpretation: The sample standard deviation 2350.6905263250064

Lab 7.1.1

```
In [ ]: import numpy as np

x = [75408, 82368, 103968, 82368, 42996, 85992, 67464, 103716, 134292, 113484,
     94848, 101628, 119712, 61776, 122496, 57720, 119712, 119712, 61140, 103716,
     67464, 58680, 91020, 83268, 62712, 78828, 82788, 91020, 75408, 59820,
     79812, 63348, 122496, 72744, 98628, 103968, 103968, 131664, 115788, 76248,
     119712, 83268, 75408, 75408, 95172, 67824, 75408, 119712, 141696, 97392,
     83268, 59820, 97668, 82368, 53736, 65676, 103716, 94848, 79020, 67824,
     86856, 103716, 75408, 119712, 91944, 94848, 66336, 99480, 119712]

np.quantile(x,[.10,.25,.50,.75,.90])
```

Out[]: array([61648.8, 75408. , 85992. , 103716. , 119712.])

Interpretation: The 10th, 25th, 50th, 75th and 90th percentile are 61648.8, 75408.0, 85992.0, 103716.0, and 119712.0 respectively.

Lab 7.2.1

```
In [ ]: import numpy as np
        from scipy import stats

x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
     60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
     72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
     82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
     88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
     103764, 105420, 109620, 146868]

x_array = np.array(x)  # convert the list to a numpy array

stats.percentileofscore(x_array,80000,kind='rank')
```

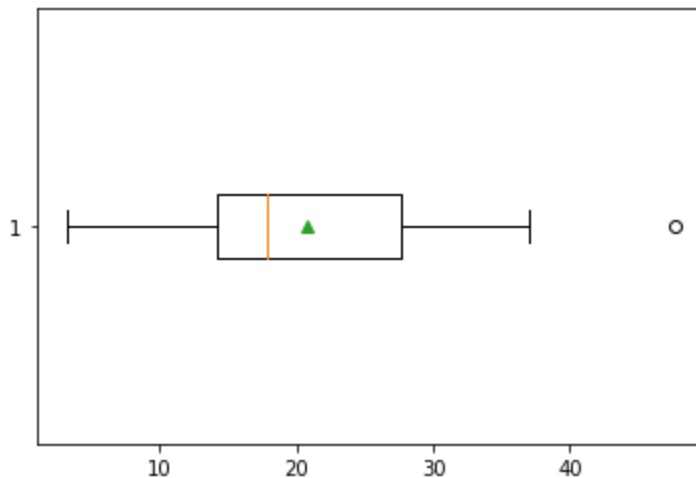
Out[]: 50.0

Interpretation: The percentile salary rank of an employee with salary 80000 is 50.

Lab 7.3.1

```
In [ ]: import matplotlib.pyplot as plt
x = [27.3, 5.5, 14.3, 3.2, 16.8, 17.9, 14.2, 14.2, 9.6, 34.7, 8.9, 30.1,
      27.7, 17.7, 21.8, 47.7, 37, 24.9]
plt.boxplot(x,vert=False,showmeans=True)
```

```
Out[ ]: {'boxes': [<matplotlib.lines.Line2D at 0x7fdd00f4be90>],
'caps': [<matplotlib.lines.Line2D at 0x7fdd00ed1f10>,
<matplotlib.lines.Line2D at 0x7fdd00ed7490>],
'fliers': [<matplotlib.lines.Line2D at 0x7fdd00ede490>],
'means': [<matplotlib.lines.Line2D at 0x7fdd00ed7f50>],
'medians': [<matplotlib.lines.Line2D at 0x7fdd00ed7a10>],
'whiskers': [<matplotlib.lines.Line2D at 0x7fdd00ed1490>,
<matplotlib.lines.Line2D at 0x7fdd00ed19d0>]}
```



Lab 8.1.1

```
In [ ]: import numpy as np

x = [0, 1, 2, 3, 4, 5, 6, 7, 8]
p = [0.197, 0.212, 0.255, 0.182, 0.09, 0.04, 0.02, 0.003, 0.001]

average = np.average(x, weights=p)
std = np.sqrt(np.cov(x, aweights=p, ddof=0))

print('mean is', average)
print('standard deviation is', std)
```

```
mean is 1.977
standard deviation is 1.5272429407268509
```

Lab 8.2.1

```
In [ ]: from scipy import stats
stats.binom.pmf(1,30,.21)
```

```
Out[ ]: 0.006769026228180294
```

```
In [ ]: from scipy import stats
stats.binom.cdf(4,30,.21)
```

```
Out[ ]: 0.2145879371696642
```

```
In [ ]: from scipy import stats
1-stats.binom.cdf(3,30,.21)
```

```
Out[ ]: 0.9015599620024068
```

Interpretation:

$$P(x = 1) = 0.006769026228180294$$

$$P(x \leq 4) = 0.2145879371696642$$

$$P(x \geq 4) = 0.7854120628303358$$

Mean = 168.0 and std = 11.520416659131735

Lab 8.2.2

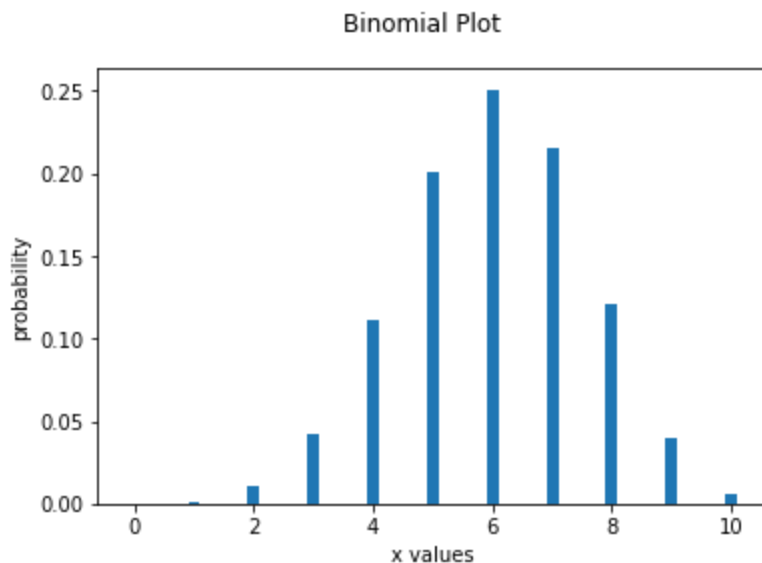
```
In [ ]: # binomial graph with n and p
def binomplot(n,p):
    from scipy import stats
    import numpy as np
    import matplotlib.pyplot as plt

    x = np.arange(0,n+1)
    y = stats.binom.pmf(x,n,p)
    plt.bar(x,y,width=0.2)

    plt.suptitle("Binomial Plot")
    plt.xlabel("x values")
    plt.ylabel("probability")

    return plt.show()
```

```
In [ ]: binomplot(10,0.6)
```



Lab 9.1.1

```
In [ ]: from scipy import stats
stats.uniform.cdf(15,5,15-5)
```

Out[]: 1.0

Lab 9.2.1

```
In [ ]: from scipy import stats
stats.norm.cdf(1.5,0,1)
```

Out[]: 0.9331927987311419

Lab 9.2.2

```
In [ ]: from scipy import stats
stats.norm.cdf(27726,27191,7126)
```

Out[]: 0.5299233487082249

Interpretation: Probability that the average cost is less than \$27726 is 0.5299233487082249

$$P(X < 27726) = 0.5299233487082249$$

Lab 9.2.3

```
In [ ]: from scipy import stats
stats.norm.ppf(.75,27191,7126)
```

```
Out[ ]: 31997.413959897276
```

Interpretation: 75 percent of the average college costs are less than or equal to 31997.413959897276

Lab 9.3.1

```
In [ ]: from scipy import stats
stats.norm.cdf(6,5,1/(10**.5))
```

```
Out[ ]: 0.9992172988709987
```

Interpretation: The probability that the mean length is less than 6 is 0.9992172988709987. That is $P(\bar{x} < 6) = 0.9992172988709987$

```
In [ ]: from scipy import stats
stats.norm.cdf(6,5,1/(10**.5)) - stats.norm.cdf(5,5,1/(10**.5))
```

```
Out[ ]: 0.49921729887099875
```

Interpretation: The probability that the average length is between 5 and 6 is 0.49921729887099875. That is, $P(5 < \bar{x} < 6) = 0.49921729887099875$

Lab 10.1.1

```
In [ ]: # Confidence interval, given confidence level C, std, n
def conf_z_mean(C,mean,std,n):
    from scipy import stats
    ebm = (std/n**.5)*stats.norm.ppf(C+(1-C)/2,0,1)
    return print("(", mean-ebm, ", ", mean+ebm, ")")
```

```
In [ ]: conf_z_mean(.90,80,15,50)
```

```
( 76.51073853896997 , 83.48926146103003 )
```

conf_z_mean(.90,80,15,50) Interpretation: The 90% confidence interval is.

Lab 10.2.1

```
In [ ]: # margin of error given the level of confidence C, mean, std and n
def error_t_mean(C,std,n):
    from scipy import stats
    ebm = (std/n**.5)*stats.t.ppf(C+(1-C)/2,n-1,0,1)
    return print("Margin of Error is",ebm)
```

```
In [ ]: error_t_mean(.9,7,25)
```

Margin of Error is 2.3952349118731986

```
In [ ]: # confidence interval given level of confidence C and x
def conf_t_mean(C,mean,std,n):
    from scipy import stats
    ebm = (std/n**.5)*stats.t.ppf(C+(1-C)/2,n-1,0,1)
    print ('mean is',mean,'and std is',std)
    return print("(", mean-ebm, ",", mean+ebm, ')')
```

```
In [ ]: conf_t_mean(.9,85,7,25)
```

mean is 85 and std is 7
(82.6047650881268 , 87.3952349118732)

Lab 10.3.1

```
In [ ]: # confidence interval and margin of error given p and n
def conf_prop_stats(C,p,n):
    from scipy import stats
    eb1 = ((p*(1-p)/n)**.5)*stats.norm.ppf(C+(1-C)/2,0,1)
    return print(" Sample Proportion =",p,"\n Margine of Error = ", eb1,
                "\n Confidence Interval",("(" ,p - eb1,"",p + eb1,")")
```

```
In [ ]: p=51/120
```

```
In [ ]: conf_prop(.9,p,120)
```

Sample Proportion = 0.425
Margine of Error = 0.07422753204362957
Confidence Interval (0.3507724679563704 , 0.4992275320436296)

Lab 10.3.2

```
In [ ]: # confidence interval and margin of error given p and n
def conf_prop(C,p,n):
    from scipy import stats
    eb1 = ((p*(1-p)/n)**.5)*stats.norm.ppf(C+(1-C)/2,0,1)
```



```
return print(" Sample Proportion =",p,"\n Margine of Error = ", eb1,
            "\n Confidence Interval", "(" ,p - eb1, ", ",p + eb1, ")")
```

```
In [ ]: conf_prop(.95,.5,120)
```

```
Sample Proportion = 0.5
Margine of Error = 0.08945970718585784
Confidence Interval ( 0.41054029281414217 , 0.5894597071858578 )
```

Lab 10.3.3

```
In [ ]: def samp_pro(C,p,E):
        from scipy import stats
        sam = p*(1-p)*stats.norm.ppf(C+(1-C)/2,0,1)**2/E**2
        return print("sample size = ", sam)
```

```
In [ ]: samp_pro(.9,.55,.03)
```

```
sample size = 744.02444498762386
```

Interpretation: A sample of size 745 is needed.

Lab 11.1.1

```
In [ ]: # z test with alternative 'smaller' or 'greater' or 'two-sided'
def z_test_mean(null_mean,sigma,sample_mean,n,alternative='two-sided'):
    from scipy import stats
    z = (sample_mean-null_mean)/(sigma/n**.5)
    if alternative == 'smaller':
        pv = stats.norm.cdf(z,0,1)
    elif alternative == 'greater':
        pv = 1-stats.norm.cdf(z,0,1)
    elif alternative == 'two-sided':
        if z < 0:
            pv = 2*stats.norm.cdf(z,0,1)
        else: pv = 2*(1-stats.norm.cdf(z,0,1))
    else:
        print("alternative option error")
    return print("z test statistic = ", z, "and p-value = ",pv)
```

```
In [ ]: z_test_mean(2,.5,2.0,25,'smaller')
```

```
z test statistic = 0.0 and p-value = 0.5
```

Interpretation: Since the p-value 0.5 is not less than alpha, fail to reject the null hypothesis.
There is not enough evidence to support the official's claim.

lab 11.2.1

```
In [ ]: import numpy as np

x=[2.5, 3.0, 1.5, 2.4, 2.6, 2.1, 1.8, 1.5, 1.7, 1.9, 3.2, 2.3, 2.0, 1.8, 1.2,
    1.5, 2.3, 2.5, 1.8, 2.2, 3.3, 2.3, 1.6, 2.5, 2.2]

mean = np.mean(x)
std = np.std(x,ddof=1)
```

```
In [ ]: t_test_mean(1.5,mean,std,25,'two-sided')
```

t test statistic = 6.006870493491006 p-value = 3.3502350138547854e-06

Interpretation: Since the p-value is 3.3502350138547854e-06 less than alpha, reject the null hypothesis. There is enough evidence to not support the official's claim.

Lab 12.3.1

```
In [ ]: def z_test_prop(null_prop,x,n,alternative): #altenative can only be "smaller"
                                                # or "greater" or "two-sided"

    from scipy import stats
    p = x/n
    z = (p-null_prop)/(null_prop*(1-null_prop)/n)**.5
    if alternative == 'smaller':
        pv = stats.norm.cdf(z,0,1)
    elif alternative == 'greater':
        pv = 1-stats.norm.cdf(z,0,1)
    elif alternative == 'two-sided':
        if z < 0:
            pv = 2*stats.norm.cdf(z,0,1)
        else: pv = 2*(1-stats.norm.cdf(z,0,1))
    else:
        print("altrnative option syntax error")
    return print("z test statistic = ", z, " p-value = ",pv)
```

```
In [ ]: z_test_prop(.5,510,1000,'greater')
```

z test statistic = 0.6324555320336764 p-value = 0.2635446284327688

Interpretation: The p-value 0.2635446284327688 is NOT less than 5%. Do not reject the null hypothesis. There is no enough evidence to support the president's claim.

Lab 12.1.1

```
In [ ]: import numpy as np

x=[1,0,3,5,14,7,4,16,9,10]
y=[17000,20000,15000,10000,5000,9000,10000,2000,8000,9000]
```

```
np.corrcoef(x,y)
```

```
Out[ ]: array([[ 1.          , -0.93825489],
               [-0.93825489,  1.          ]])
```

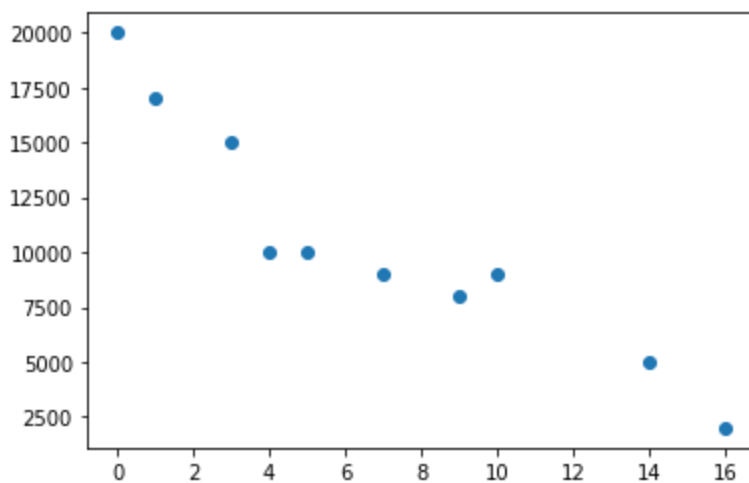
Interpretation: The correlation coefficient is -0.93825489

```
In [ ]: import matplotlib.pyplot as plt

x=[1,0,3,5,14,7,4,16,9,10]
y=[17000,20000,15000,10000,5000,9000,10000,2000,8000,9000]

plt.scatter(x,y)
```

```
Out[ ]: <matplotlib.collections.PathCollection at 0x7fa3d949c190>
```



Lab 12.3.1

```
In [ ]: from scipy import stats

x=[1,0,3,5,14,7,4,16,9,10]
y=[17000,20000,15000,10000,5000,9000,10000,2000,8000,9000]

stats.linregress(x,y)
```

```
Out[ ]: LinregressResult(slope=-955.6247567146751, intercept=17093.81082133126, rvalue=-0.9382548923679298, pvalue=5.899842619735838e-05, stderr=124.57425008729852)
```

Interpretation: slope=-955.6247567146751, intercept=17093.81082133126, rvalue=-0.9382548923679298, pvalue=5.899842619735838e-05

Lab 12.3.1 (Alternative Solution Using Array)

```
In [ ]: from scipy import stats

x=[1,0,3,5,14,7,4,16,9,10]
```

```

y=[17000,20000,15000,10000,5000,9000,10000,2000,8000,9000]

results = stats.linregress(x,y) # save results as an array named results

print('slope =', results[0]) # results[0] is the slope
print('intercept =', results[1]) # results[1] is the intercept
print('r =', results[2]) # results[2] is the r correlation-coefficient
print('p-value =', results[3]) # results[3] is p-value for testing pop. cor. co

```

```

slope = -955.6247567146751
intercept = 17093.81082133126
r = -0.9382548923679298
p-value = 5.899842619735838e-05

```

Interpretation: slope=-955.6247567146751, intercept=17093.81082133126,
 rvalue=-0.9382548923679298, pvalue=5.899842619735838e-05

```

In [ ]: import matplotlib.pyplot as plt
import numpy as np

x=[1,0,3,5,14,7,4,16,9,10]
y=[17000,20000,15000,10000,5000,9000,10000,2000,8000,9000]

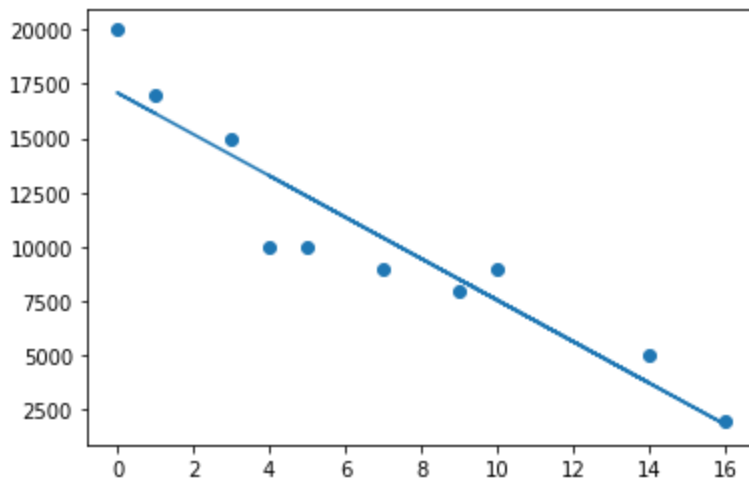
x = np.array(x)
y = np.array(y)

y_hat = -955.6247567146751*x+17093.81082133126

plt.scatter(x,y)
plt.plot(x,y_hat)

```

Out[]: [matplotlib.lines.Line2D at 0x7fa3d9247f90]



Lab 13.1.1

```

In [ ]: import numpy as np
from scipy import stats

```

```
obs = [306,232]
sum = np.sum(obs)
exp = [.513*sum,.469*sum]

stats.chisquare(obs,exp)
```

Out[]: Power_divergenceResult(statistic=4.89897772395255, pvalue=0.026872598995409867)

Interpretation: Since the p-value 0.026872598995409867 is less than $\alpha = .10$, reject the null-hypothesis. There is enough evidence to reject the official's claim.

Lab 13.2.1

```
In [ ]: def chi2_test_of_indp(obs):
        from scipy import stats
        chi2, p, dof, ex = stats.chi2_contingency(obs)

        print("chi-square test statistic:",chi2)
        print("p-value:", p)
```

```
In [ ]: r1 = [1734,475,1281]
        r2 = [2071,373,1938]
        r3 = [678,103,935]
        r4 = [534,92,560]
        r5 = [5644,1661,2613]
        r6 = [4094,927,1277]

        obs = [r1,r2,r3,r4,r5,r6]

        chi2_test_of_indp(obs)
```

chi-square test statistic: 1509.7409528009978
p-value: 0.0

Interpretation: Since the p-value=0.0 is less than the level of significance, reject the null hypothesis. That is, there is enough evidence that race and crime-class are NOT independent.

Lab 14.1.1

```
In [ ]: import pandas as pd

        x = ['male','female','female','male','male']
        x_pd = pd.DataFrame({'gender':x})
        print(x_pd)
```

```

gender
0    male
1  female
2  female
3    male
4    male

```

Lab 14.1.2

```

In [ ]: import pandas as pd
x=[20, 21, 25, 20, 29, 25, 22, 25, 21, 25]
x_df=pd.DataFrame({'data':x})
x_df.value_counts(sort=False)

```

```

Out[ ]: data
20      2
21      2
22      1
25      4
29      1
dtype: int64

```

data	freq
20	2
21	2
22	1
25	4
29	1

Lab 14.1.3

```

In [ ]: import pandas as pd
x=[20, 21, 25, 20, 29, 25, 22, 25, 21, 25]
x_df=pd.DataFrame({'data':x})
x_df.value_counts(sort=False,normalize=True)

```

```

Out[ ]: data
20      0.2
21      0.2
22      0.1
25      0.4
29      0.1
dtype: float64

```

Lab 14.1.4

```
In [ ]: import pandas as pd
x=['NO', 'NO', 'YES', 'YES', 'NO', 'YES', 'NO', 'NO', 'YES', 'NO', 'YES', 'NO',
   'NO', 'YES', 'YES', 'YES', 'YES', 'NO', 'YES', 'YES']

x_df = pd.DataFrame({'vote':x})
x_df.value_counts(normalize=True)
```

```
Out[ ]: vote
YES      0.55
NO       0.45
dtype: float64
```

Lab 14.1.5

```
In [ ]: import pandas as pd
x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
     60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
     72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
     82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
     88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
     103764, 105420, 109620, 146868]

x_df = pd.DataFrame({'data':x})
x_df.describe()
```

```
Out[ ]:      data
count    54.000000
mean     78460.000000
std      18253.562584
min       46776.000000
25%      64377.000000
50%      78366.000000
75%      88272.000000
max      146868.000000
```

Lab 14.1.6

```
In [ ]: import pandas as pd
x = [46776, 53736, 53736, 54840, 57408, 57408, 57408, 57408, 57408, 57408,
     60108, 60108, 62712, 63552, 66852, 66852, 69732, 70032, 70044, 70272,
```

```
72024, 73380, 73380, 73380, 75408, 75468, 76248, 80484, 80484, 80484,
82476, 82788, 82836, 83676, 84324, 84324, 84324, 85848, 87564, 88272,
88272, 90828, 92004, 92304, 92520, 96060, 96096, 100680, 100716, 100716,
103764, 105420, 109620, 146868]
```

```
x_df = pd.DataFrame({'salary':x})
print('skewness index is ',x_df.skew())
print('kurtosis index is',x_df.kurtosis())
```

```
skewness index is salary    0.924462
dtype: float64
kurtosis index is salary    2.250167
dtype: float64
```

Lab 14.2.1

```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data/valid_import.csv'

x_df = pd.read_csv(url)

x_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 120 entries, 0 to 119
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   valid_import          120 non-null   bool
1   item                  120 non-null   object
2   importer_id           120 non-null   int64
3   exporter_id           120 non-null   int64
4   country_of_origin     120 non-null   object
5   declared_quantity     120 non-null   int64
6   declared_cost         120 non-null   float64
7   mode_of_transport     120 non-null   object
8   route                 120 non-null   object
9   date_of_departure     120 non-null   object
10  date_of_arrival       120 non-null   object
11  declared_weight       120 non-null   float64
12  actual_weight         120 non-null   float64
13  days_in_transit       120 non-null   float64
dtypes: bool(1), float64(4), int64(3), object(6)
memory usage: 12.4+ KB
```

Interpretation: for x_df

1. class type is Pandas Dataframe
2. 120 entries from 0 to 199
3. 14 Columns
4. Info for each Column listed

Lab 14.3.1

```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data/imports_2015.csv'

x_df = pd.read_csv(url)

x_df.describe()
```

```
Out[ ]:
```

	importer_id	exporter_id	declared_quantity	declared_cost	declared_weight	actual_weight
count	120.0	120.0	120.000000	120.000000	120.000000	120.000000
mean	111.0	222.0	127.458333	6743.649881	1264.702934	1306.400000
std	0.0	0.0	14.641311	2991.797050	633.149971	656.900000
min	111.0	222.0	100.000000	1441.012419	18.459509	19.200000
25%	111.0	222.0	115.750000	4442.903914	820.314400	841.700000
50%	111.0	222.0	131.500000	6010.218745	1255.597743	1305.700000
75%	111.0	222.0	139.000000	8887.095370	1711.314045	1763.600000
max	111.0	222.0	149.000000	14281.325362	2806.338955	2918.600000

Interpretation: The descriptive statistics for `importer_id` and `exporter_id` are meaningless and should be ignored.

Lab 14.3.2

```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data/imports_2015.csv'

x_df = pd.read_csv(url)

x_df.groupby(['country_of_origin']).agg({'actual_weight': ['mean', 'std', 'size']})
```

Out[]:

	actual_weight		
	mean	std	size
country_of_origin			
China	1307.147246	627.380399	24
France	1218.459074	728.100019	24
India	1304.722531	493.478451	24
Italy	1190.657039	711.818853	24
USA	1511.163138	702.116897	24

Lab 14.3.3

```
In [ ]: import pandas as pd

url = 'https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data/actual_weight.csv'

x_df = pd.read_csv(url)

x_df.groupby(['country_of_origin', 'route']).agg({'actual_weight': ['mean', 'std', 'size']})
```

Out[]:

		actual_weight		
		mean	std	size
country_of_origin	route			
China	america	1281.081635	873.244407	6
	asia	1492.969412	577.826149	6
	europa	1155.582715	465.171493	6
	panama	1298.955222	654.115536	6
France	america	1142.095755	686.692379	6
	asia	1218.311778	936.931509	6
	europa	1062.014711	756.490150	6
	panama	1451.414052	644.548543	6
India	america	1007.777832	495.847993	6
	asia	1697.493097	254.063822	6
	europa	1379.484100	516.208468	6
	panama	1134.135095	459.075799	6
Italy	america	1035.376749	923.534914	6
	asia	1512.269050	974.452899	6
	europa	1002.642317	461.640254	6
	panama	1212.340040	345.152024	6
USA	america	1352.550563	895.589850	6
	asia	1730.803390	725.189852	6
	europa	1445.528489	666.483253	6
	panama	1515.770109	634.190270	6

Lab 14.3.4

In []:

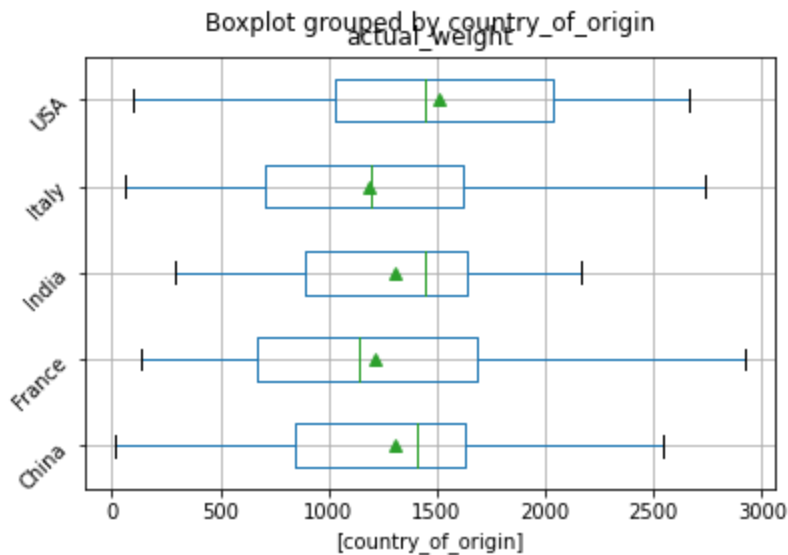
```
import pandas as pd

url = 'https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/data/actual_weight.csv'

x_df = pd.read_csv(url)

x_df.boxplot(column=['actual_weight'],by=['country_of_origin'],showmeans=True,vert=
```

Out[]: <matplotlib.axes._subplots.AxesSubplot at 0x7f98bff49a90>



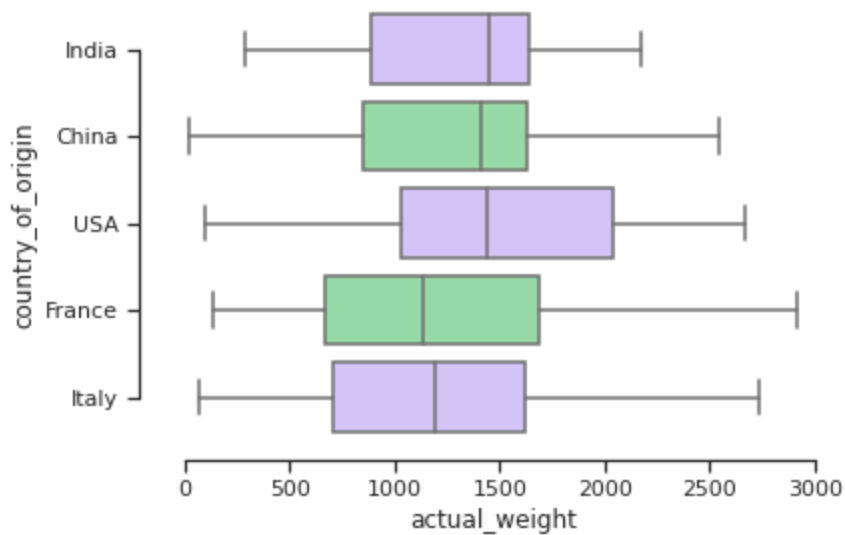
Lab 14.3.4 Alternative Solution

```
In [2]: import pandas as pd
import seaborn as sns
sns.set_theme(style="ticks", palette="pastel")

# Load the example tips dataset
url = 'https://raw.githubusercontent.com/swaathi/cleaning-datasets-using-pandas/master/tips.csv'

x_df = pd.read_csv(url)

sns.boxplot(x="actual_weight", y="country_of_origin", hue_order="smoker", palette=[
sns.despine(offset=10, trim=True)
```



Lab 14.4.1

```
In [ ]: import pandas as pd

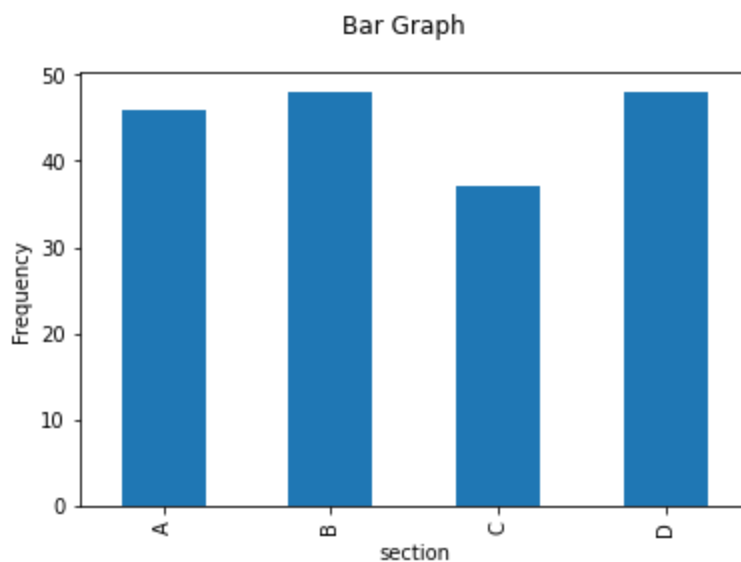
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
x_df.groupby(['section']).size().plot(kind='bar')

import matplotlib.pyplot as plt

plt.suptitle("Bar Graph")
plt.ylabel("Frequency")
```

Out[]: Text(0, 0.5, 'Frequency')



Lab 14.4.2

```
In [ ]: import pandas as pd

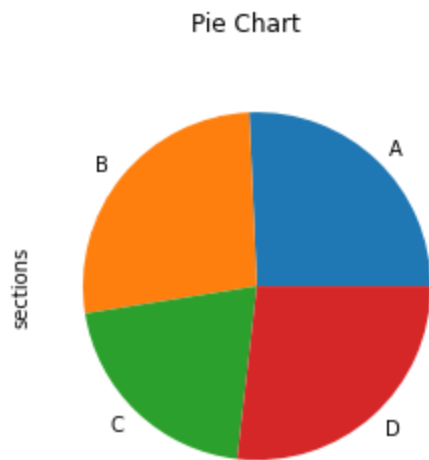
url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
x_df.groupby(['section']).size().plot(kind='pie')

import matplotlib.pyplot as plt

plt.suptitle("Pie Chart")
plt.ylabel("sections")
```

Out[]: Text(0, 0.5, 'sections')



Lab 14.5.1

```
In [ ]: import pandas as pd
```

```
url = 'https://raw.githubusercontent.com/Shreyas3108/house-price-prediction/master/  
x_df = pd.read_csv(url)
```

```
In [ ]: import statsmodels.formula.api as smf  
model = smf.ols(formula='price ~ bedrooms + sqft_basement + sqft_living', data=x_df  
results = model.fit()  
  
print(results.summary())
```

OLS Regression Results

=====						
Dep. Variable:	price	R-squared:	0.508			
Model:	OLS	Adj. R-squared:	0.508			
Method:	Least Squares	F-statistic:	7426.			
Date:	Wed, 11 Aug 2021	Prob (F-statistic):	0.00			
Time:	13:38:08	Log-Likelihood:	-2.9995e+05			
No. Observations:	21613	AIC:	5.999e+05			
Df Residuals:	21609	BIC:	5.999e+05			
Df Model:	3					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	8.547e+04	6673.321	12.807	0.000	7.24e+04	9.85e+04
bedrooms	-5.806e+04	2312.155	-25.111	0.000	-6.26e+04	-5.35e+04
sqft_basement	26.6854	4.409	6.053	0.000	18.044	35.327
sqft_living	308.9346	2.478	124.668	0.000	304.077	313.792
=====						
Omnibus:	14369.246	Durbin-Watson:	1.985			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	489010.614			
Skew:	2.718	Prob(JB):	0.00			
Kurtosis:	25.660	Cond. No.	9.07e+03			
=====						

Warnings:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 9.07e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Interpretation:

Interpretation:

1. See `x_df.head()` above
2. $\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3x_3$ where
 $\hat{y} = 8.547e04 - 5.806e04b_1 + 26.6854b_2 + 309.9346b_3$
3. The F-statistic is 7426 with p-value of Prob (F-statistic) 0.00. Thus, the model has a significant contribution to the prediction of price.
4. The coefficient of determination (R-squared:) is 0.508.
5. The p-value (P>|t|) for the `sqft_living` is 0.000 which is less than $\alpha = 0.05$. It is significant.

Lab 14.5.2

```
In [ ]: import pandas as pd
df = pd.read_csv('https://raw.githubusercontent.com/simeonaman/demo/master/heart.csv')
df.head()
```

```
Out[ ]:   row.names  sbp  tobacco  ldl  adiposity  famhist  typea  obesity  alcohol  age  chd
0         1  160    12.00  5.73    23.11  Present    49    25.30    97.20  52    1
1         2  144     0.01  4.41    28.61  Absent    55    28.87     2.06  63    1
2         3  118     0.08  3.48    32.28  Present    52    29.14     3.81  46    0
3         4  170     7.50  6.41    38.03  Present    51    31.99    24.26  58    1
4         5  134    13.60  3.50    27.78  Present    60    25.99    57.34  49    1
```

```
In [ ]: import statsmodels.formula.api as smf
model = smf.logit(formula='chd ~ ldl + age + sbp', data=df)
results = model.fit()

print(results.summary())
```

Optimization terminated successfully.

Current function value: 0.553757

Iterations 6

Logit Regression Results

```
=====
Dep. Variable:          chd  No. Observations:          462
Model:                Logit  Df Residuals:            458
Method:                 MLE  Df Model:                3
Date:                  Wed, 11 Aug 2021  Pseudo R-squ.:         0.1416
Time:                  14:09:16  Log-Likelihood:        -255.84
converged:              True  LL-Null:              -298.05
Covariance Type:        nonrobust  LLR p-value:           3.427e-18
=====
               coef      std err          z      P>|z|      [0.025      0.975]
-----
Intercept    -4.7372      0.768      -6.170      0.000      -6.242      -3.233
ldl           0.1863      0.053       3.487      0.000       0.082       0.291
age           0.0559      0.009       6.036      0.000       0.038       0.074
sbp           0.0048      0.005       0.898      0.369      -0.006       0.015
=====
```

Interpretation:

$$1. \hat{p} = \frac{1}{1 + e^{-4.7372 + 0.1863ldl + 0.0559age + 0.0048sbp}}$$

2. Since LLR p-value 3.427e-18 is less than level of significance, the model contributes significantly to the prediction of **chd**.

Lab 14.6.1


```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.groupby(['level']).agg({'test3_score': ['mean', 'std', 'size']})
```

```
Out[ ]:
```

	test3_score		
	mean	std	size
level			
high	53.773333	3.184170	60
low	53.750000	2.329727	60
medium	53.831667	2.536546	60

```
In [ ]: import statsmodels.api as sm
from statsmodels.formula.api import ols
model = ols('test3_score ~ C(level)', data = x_df).fit()
table = sm.stats.anova_lm(model, type=1) # type 2 for equal or non-equal sample size
print(table)
```

	df	sum_sq	mean_sq	F	PR(>F)
C(level)	2.0	0.212333	0.106167	0.014477	0.985629
Residual	177.0	1298.037167	7.333543	NaN	NaN

Interpretation: Since the p-value 0.985629 is not less than $\alpha = 0.10$, we fail to reject the null hypothesis that the averages for all three levels are equal. That is, there is no significant difference between the averages of `test3_score` of the three levels.

Lab 14.6.2

```
In [ ]: import pandas as pd

url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

x_df = pd.read_csv(url)
```

```
In [ ]: x_df.groupby(['level', 'gender']).agg({'test3_score': ['mean', 'std', 'size']})
```

Out[]:

		test3_score		
		mean	std	size
level	gender			
high	female	54.036000	3.487463	25
	male	53.585714	2.986946	35
low	female	53.857576	2.521412	33
	male	53.618519	2.111696	27
medium	female	53.604167	2.688863	24
	male	53.983333	2.456769	36

```
In [ ]: import statsmodels.api as sm
        from statsmodels.formula.api import ols
        model = ols('test3_score ~ C(level) + C(gender)', data = x_df).fit()
        table = sm.stats.anova_lm(model, type=1) # type 2 for non-equal sample sizes
        print(table)
```

	df	sum_sq	mean_sq	F	PR(>F)
C(level)	2.0	0.212333	0.106167	0.014401	0.985704
C(gender)	1.0	0.494704	0.494704	0.067102	0.795906
Residual	176.0	1297.542462	7.372400	NaN	NaN

Interpretation: There is no significant difference of the `test3_score` means between the three levels after blocking by gender. No significant difference in the average test1_scores between the two genders.

Lab 14.6.3

```
In [ ]: import pandas as pd

        url='https://raw.githubusercontent.com/simeonaman/demo/master/my_csv_file.csv'

        x_df = pd.read_csv(url)
        x_df.head()
```

Out[]:

	id	gender	section	time	level	test1_score	test2_score	test3_score
0	1	female	C	17.3	low	65.4	84.3	51.9
1	2	female	D	5.1	low	72.4	86.1	52.0
2	3	female	A	2.4	low	73.8	84.2	53.1
3	4	female	A	8.8	low	70.2	87.9	52.0
4	5	male	B	15.0	low	60.3	81.4	55.5

In []: `x_df.groupby(['level','section']).agg({'test3_score': ['mean', 'std','size']})`

Out[]:

		test3_score		
		mean	std	size
level	section			
high	A	54.207143	3.816657	14
	B	52.538462	2.417484	13
	C	54.146667	3.044636	15
	D	54.105882	3.362192	17
low	A	54.000000	1.942384	15
	B	53.618750	1.880858	16
	C	53.750000	2.469548	16
	D	53.623077	3.181235	13
medium	A	53.982353	2.444441	17
	B	53.126316	1.659264	19
	C	53.250000	2.962938	6
	D	54.627778	3.136841	18

In []: `from statsmodels.formula.api import ols
model = ols('test3_score ~ C(level)*C(section)',data = x_df).fit()
table = sm.stats.anova_lm(model,type=2) # type 2 for non-equal sample sizes
print(table)`

	df	sum_sq	mean_sq	F	PR(>F)
C(level)	2.0	0.200742	0.100371	0.013460	0.986631
C(section)	3.0	31.217036	10.405679	1.395460	0.246013
C(level):C(section)	6.0	19.884361	3.314060	0.444434	0.848150
Residual	167.0	1245.286911	7.456808	NaN	NaN

Interpretation: There is no significant interaction effect between **level** and **section** of the average **test2_score** . 0.848150 > 0.10

References

Arithmetic Operators

Name	Operator	Example	Result
Addition	+	5+6	11
Subtraction	-	5-6	-1
Multiplication	*	5*6	30
Division	/	5/6	0.83333
Exponentiation	**	5**2	5 ² = 25

Descriptive Stats

```
import numpy as np
import pandas as pd

x=[x1,x2,x3,...]

x_df=pd.DataFrame({'column_name':x})
```

procedure	numpy	pandas
frequency counts	np.unique(x,return_counts=True)	x_df.value_counts()
size	np.size(x)	x_df.count()
min	np.amin(x)	x_df.min()
max	np.amax(x)	x_df.max()
mean	np.mean(x)	x_df.mean()
mode	NA	x_df.mode()
median	np.median(x)	x_df.median()
sample variance	np.var(x,ddf=1)	x_df.var()
pop variance	np.var(x)	x_df.var(ddof=0)
sample std	np.std(x,ddf=1)	x_df.std()
pop std	np.std(x)	x_df.std(ddof=0)
weighted mean	np.average(x,weights=freq)	NA

procedure	numpy	pandas
weighted sample std	<code>(np.cov(x, fweights=freq, ddof=1))**(1/2)</code>	NA
weighted pop std	<code>(np.cov(x, fweights=freq))**(1/2)</code>	NA
percentiles	<code>np.quantile(x, p)</code>	<code>x_df.quantile(p)</code>
summary stats	NA	<code>x_df.describe()</code>
factorial, n!	<code>np.math.factorial(n)</code>	NA
n random integers from a to b	<code>np.random.randint(a, b+1, size=n)</code>	NA
weighted mean	<code>np.average(x, weights=y)</code>	NA
Weighted std	<code>(np.cov(x, aweights=y, ddof=0))**(1/2)</code>	NA
correlation coef.	<code>np.corrcoef(x, y)</code>	<code>x_df.corr()</code>

NA = Not Available

Summary statistics by grouping with categorical columns:

```
x_df.groupby(['level', 'gender']).agg({'test1_score':
['mean', 'std', 'size']})
```

Graphs

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
x = [x1, x2, x3, ...]
x_df = pd.DataFrame({'column_name', x})
```

CHOOSE A CODE FROM BELOW (Bar, Pie, Histogram etc)

```
plt.suptitle("Plot Title")
plt.xlabel("x-axis label")
plt.ylabel("y-axis label")
```

1. Bar graph (raw data)

```
x_df.groupby(['column_name']).size().plot(kind='bar')
```

2. Pie Chart (raw data) `x_df.groupby(['column_name']).size().plot(kind='pie', autopct='%1.1f%%')`

3. Bar graph (frequency table) `x_df.plot.bar(x='column_name', y='frequency')`

4. Pie chart (frequency table)

```
x_df.set_index('column_name',inplace=True)
x_df.plot.pie(y='frequency')
```

5. Side by Side Bar (frequency table)

```
x_df.set_index('column_name',inplace=True)
x_df.plot.bar()
```

6. Histogram

```
freq,bins,patches = plt.hist(x,bins=5,ec='k',range=
[ min(x),max(x)+1])
plt.xticks(bins)    # df['column_name']=x
```

7. Scater Plot (x, y list given) `plt.scatter(x,y)`

Linear Regression

```
from scipy import stats
stats.linregress(x_list, y_list)
```

Multiple Linear Regression

```
import statsmodels.formula.api as smf
model = smf.ols(formula='y ~ x1 + x2 +...+ xk', data=x_df)
results = model.fit()

print(results.summary())
```

Probability Distributions

```
from scipy import stats
```

procedure	scipy.stats
Binomial pdf $P(X = x)$	<code>stats.binom.pmf(x,n,p)</code>
Binomial cdf $P(X \leq x)$	<code>stats.binom.cdf(x,n,p)</code>
Normal cdf $P(X \leq x)$	<code>stats.norm.cdf(x,mean,std)</code>
Inverse Normal or p-th Percentile	<code>stats.norm.ppf(p,mean,std)</code>
t distribution cdf $P(X \leq x)$	<code>stats.t.cdf(t,df,0,1)</code>
Inverse t or p-th Percentile	<code>stats.t.ppf(p,df,0,1)</code>

procedure	scipy.stats
chi-square cdf $P(\chi^2 \leq ch2)$	<code>stats.chi2.cdf(ch2,k-1)</code>
Inverse chi-square or p-th Percentile	<code>stats.chi2.ppf(p,k-1)</code>

Margin of Error for Confidence Interval

```
from scipy import stats
```

Mean μ with z distribution with confidence level `C`, (population) standard deviation `std` and sample size `n`:

```
E = (std/n**.5)*stats.norm.ppf(C+(1-C)/2,0,1)
```

Mean μ with t distribution with confidence level `C`, sample standard deviation `std` and sample size `n`:

```
E = (std/n**.5)*stats.t.ppf(C+(1-C)/2,0,1)
```

Proportion p with z distribution with confidence level `C`, sample proportion `p` and sample size `n`:

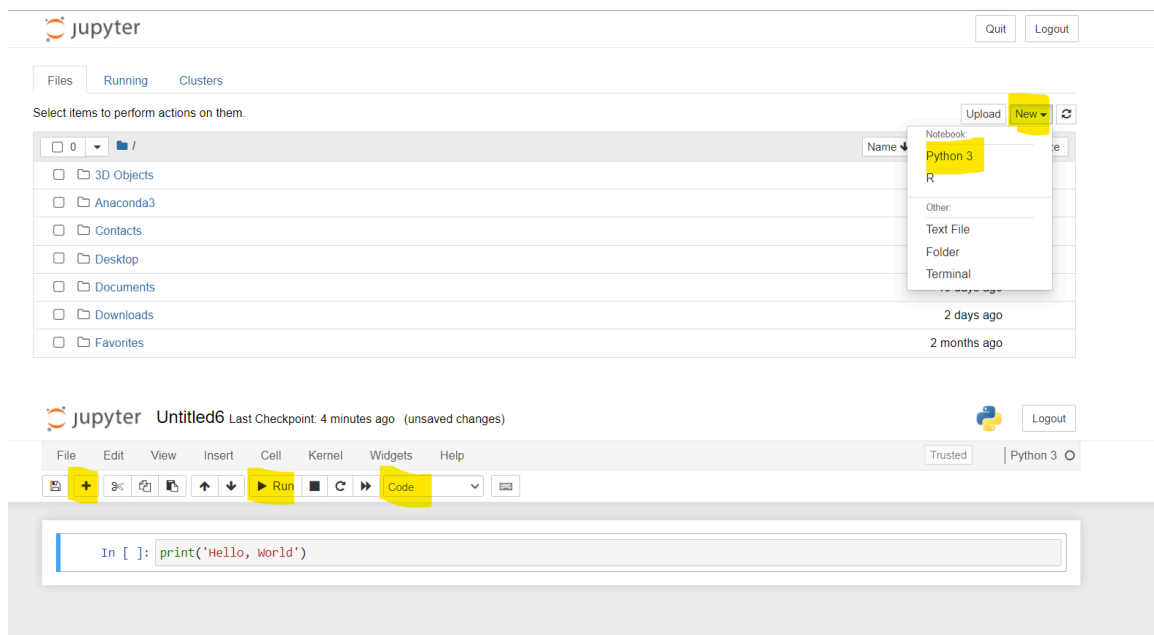
```
E = (p*(1-p)/n)**.5)*stats.norm.ppf(C+(1-C)/2,0,1)
```

Appendix 1

Python Computer Installation

If you prefer not to use Google Colab Notebook, you can install free Python in your computer and use the **Anaconda Jupyter Notebook**. Go to [this website](#) and select Windows, MacOS or Linux, depending on your computer. When the installation is completed, open **jupyter Notebook(anaconda3)** from the **Anaconda** folder on your computer. You can also watch YouTube videos on how to install Anaconda for [Windows 10](#) or [MacOS](#).

Open **Jupyter Notebook** from your Windows computer Start menu under **Anaconda** and click **new**. You can add new code-cell or text-cell by clicking + and code (see pictures below). You can run a code-cell by clicking **run**.



Appendix 2: Random Integer Generator

[Watch demo video](#)

Appendix 3: Factorials

[Watch demo video](#)

Resources

- [Python.org](#)
- [Matplotlib](#)
- [Python eBook](#)
- [Introductory Statistics eBook](#)
- [Python Tutorials](#)
- [Terminology](#)
- [Numpy](#)
- [Scipy Stats -Pandas](#)